



UNIVERSITÄT
LEIPZIG

Fakultät

Institut

Fachgebiet

Erstbetreuer

Zweitbetreuer

Thema

Titel

Bachelorarbeit zur Erlangung des akademischen Grades

Bachelor of Science – *Studiengang*

vorgelegt von: *Autorenname*

Matrikelnummer: *Matrikelnummer*

E-Mail-Adresse: *E-Mail*

Telefonnummer: *Telefonnummer*

Anschrift: *Straße*
Plz und Ort

Leipzig, den Datum

Zusammenfassung

Dein Abstract

Schlüsselwörter:

Deine Schlüsselwörter

Inhaltsverzeichnis

Inhaltsverzeichnis	I
Abbildungsverzeichnis	IV
Tabellenverzeichnis	V
Formelverzeichnis	VI
Abkürzungsverzeichnis	VII
1 Einleitung	1
1.1 Motivation und Problemstellung	1
1.2 Zielstellung	2
1.3 Aufbau der Arbeit	3
2 Fachliche Grundlagen	4
2.1 Reinforcement Learning	4
2.1.1 Markov Decision Process	4
2.2 Lösungen von MDPs	5
2.2.1 Dynamische Programmierung	5
2.2.2 Monte-Carlo- und Temporal-Difference-Verfahren	6
2.2.3 Funktionsapproximation	6
2.2.4 Policy-Gradient-Methoden	7
2.2.5 Proximal Policy Optimization	7
2.3 Convolutional Neural Networks	8
2.4 Achsenparallele Begrenzungsboxen	9
3 Stand der Forschung	11
3.1 Autonomes Landen von Drohnen	11
3.2 Tiefenbildbasierte Hindernisvermeidung	14
3.3 Reinforcement Learning für Drohnensteuerung	16
4 Methodik	20
4.1 Systemüberblick	20

4.2	Formulierung als Markov-Entscheidungsprozess	21
4.2.1	Zustandsraum	22
4.2.2	Aktionsraum	23
4.2.3	Übergangsmodell	23
4.2.4	Belohnungsfunktion	23
4.2.5	Terminierung	26
4.3	Netzwerkarchitektur	26
4.4	Algorithmus	27
4.4.1	Proximal Policy Optimization	27
4.4.1.1	Aktionsverteilung und Tanh-Squashing	28
4.4.1.2	Hyperparameter	28
4.5	Trainingsumgebung	29
4.5.1	Physiksimulator	29
4.5.2	Drohnenmodell	30
4.5.3	Korridorgeometrie	30
4.5.4	Hindernisse	30
4.6	Kaskadierender PID-Regler	31
4.7	Trainingsprozess	32
4.8	Evaluation	33
4.8.1	Evaluationsmetriken	33
4.8.2	Versuchsreihe 1: Generalisierung auf unbekannte Hindernisformen	34
4.8.3	Versuchsreihe 2: Transfer in eine realistische Agrarumgebung . . .	35
5	Ergebnisse	37
5.1	Trainingsverhalten	37
5.2	Phase 1: Korridornavigation	38
5.2.1	Nachuntersuchung: Oberflächenbasierte Kollisionserkennung . . .	40
5.3	Phase 2: Weinberg-Transfer	40
5.3.1	Sensitivitätsanalyse des Erfolgskriteriums	41
5.4	Vergleich RL vs. RRT-Connect	42
6	Diskussion	44
6.1	Leistungsfähigkeit in der Trainingsumgebung	44
6.1.1	Trainingsverhalten und Konvergenz	44

6.1.2	Dominante Fehlerart und Flugverhalten	45
6.2	Transferleistung auf den Weinberg	46
6.2.1	Leistungsabfall und Verschiebung der Fehlerarten	46
6.2.2	Einordnung als Sim-to-Sim Transfer	46
6.2.3	Perceptual Aliasing als Erklärung für die Terrain-Kollisionen	47
6.3	Leistungslücke zum Pfadplaner	49
6.3.1	Informationsasymmetrie und Erfolgsrate	49
6.3.2	Landezeit und Rechenzeit	49
6.4	Limitationen	50
6.4.1	Trainingssetup	50
6.4.2	Sensorik und Architektur	50
6.4.3	Umgebungsdiversität	51
7	Fazit und Ausblick	52
7.1	Beantwortung der Forschungsfragen	52
7.2	Limitationen der Arbeit	53
7.3	Perspektiven für zukünftige Forschung	53
7.3.1	Architektur Erweiterungen	53
7.3.2	Trainingssetup	53
7.3.3	Umgebungsdiversität und Sim-to-Real-Transfer	53
Anhang		VII
Literatur		XII
Erklärung		XXII

Abbildungsverzeichnis

2.1	Interaktion zwischen Agent und Umgebung in einem MDP (nach Sutton und Barto 2020, S. 48).	5
2.2	L^{CLIP} als Funktion des Wahrscheinlichkeitsverhältnisses r . Links ($\hat{A}_t > 0$): der Anreiz wird bei $1 + \varepsilon$ gekappt. Rechts ($\hat{A}_t < 0$): die Korrektur bleibt uneingeschränkt (nach (Schulman, Wolski u. a. 2017), Fig. 1).	8
4.1	Systemarchitektur mit zwei Steuerungsebenen: Der RL-Agent (oben) erhält Tiefenbild und Zustandsvektor, sampelt eine Aktion und gibt Sollwerte vor. Der PID-Regler (unten) setzt diese über 300 Simulationsschritte in Motordrehzahlen um.	21
4.2	Netzwerkarchitektur: Das Tiefenbild durchläuft einen geteilten CNN-Encoder (drei Faltungsschichten mit Batch-Normalisierung, ELU und abschließendem Global Max Pooling). Der resultierende 128-dimensionale Merkmalsvektor wird mit dem empirisch normalisierten Zustandsvektor konkateniert und an die getrennten MLP-Köpfe von Actor und Critic weitergegeben.	27
4.3	Seitenansichten der Korridor geometrie. Links: xz -Ebene (Slalom-Achse) mit schematischer Flugbahn. Rechts: yz -Ebene. Die gestrichelten Rechtecke markieren die Korridor Grenzen, graue Flächen die schematischen Hindernis-Begrenzungsboxen.	31
4.4	Zielpositionen in der Weinbergumgebung	36
5.1	Kumulative Belohnung und Standardabweichung der Aktionsverteilung	37
5.2	Absturzstrafe und Erfolgsbelohnung	38
5.3	Episodenergebnisse Phase 1	39
5.4	Exemplarische 3D-Trajektorien in Phase 1	40
5.5	Episodenergebnisse Phase 2	41
6.1	Perceptual Aliasing im Tiefenbild	48

Tabellenverzeichnis

3.1	Vergleich lernbasierter Systeme zur autonomen Drohnennavigation und -landung. Die Spalten <i>Hind.</i> und <i>Landung</i> kennzeichnen, ob das System aktive Hindernisvermeidung bzw. Präzisionslandung adressiert.	18
4.1	Komponenten des Zustandsvektors $\mathbf{o}_t \in \mathbb{R}^{17}$	22
4.2	Gewichte der Belohnungskomponenten. Negative Gewichte kennzeichnen Strafterme.	25
4.3	PPO-Hyperparameter der vorliegenden Arbeit.	29
4.4	Trainingsparameter.	32
5.1	Konvergenzgeschwindigkeit und Checkpoint-Selektion	38
5.2	Vergleich AABB- und oberflächenbasierte Kollisionserkennung	40
5.3	Sensitivitätsanalyse des Erfolgsradius in Phase 2	42
5.4	Vergleich RL-Agent und RRT-Connect	42

Formelverzeichnis

Abkürzungsverzeichnis

AABB		Axis-Aligned Bounding Box
CNN		Convolutional Neural Network
GNSS		Global Navigation Satellite System
GPU		Graphics Processing Unit
GSO		Google Scanned Objects
HPC		High-Performance Computing
IMU		Inertial Measurement Unit
MDP		Markov Decision Process
ML		Machine Learning
MLP		Multilayer Perceptron
PID		Proportional-Integral-Derivative
PPO		Proximal Policy Optimization
RL		Reinforcement Learning
URDF		Unified Robot Description Format

1 Einleitung

1.1 Motivation und Problemstellung

Die Landwirtschaft steht vor einer Reihe gleichzeitiger Herausforderungen: Der Klimawandel führt zu erhöhtem Schädlingsdruck, veränderten Niederschlagsmustern und sinkenden Erträgen (Calvin u. a. 2023; Hultgren u. a. 2025; *Climate Change Trends and Impacts on California Agriculture: A Detailed Review* | MDPI 2026), während eine wachsende Weltbevölkerung und zunehmender Fachkräftemangel den Produktionsdruck weiter verschärfen (Food and Agriculture Organization of the United Nations 2017; *World Population Prospects* 2026; Duckett u. a. 2018). Als Teilgebiet der digitalen Landwirtschaft (Basso und Antle 2020) bieten Drohnen (unbemannte Luftfahrzeuge) vielversprechende Lösungsansätze: Sie ermöglichen unter anderem die gezielte Ausbringung von Pflanzenschutzmitteln sowie großflächiges Crop-Monitoring mittels Remote Sensing (Mashori u. a. 2026; Radoglou-Grammatikis u. a. 2020) und können so dazu beitragen, die Landwirtschaft effizienter und resilienter zu gestalten (Unde u. a. 2025; Guebsi u. a. 2024).

Trotz umfangreicher Forschung zu Drohnensystemen bleibt autonomes Landen eine offene Herausforderung (Amendola u. a. 2024). Unfallfreies Landen ist für die meisten Flugmissionen essentiell; gleichzeitig erfordern die vielfältigen Landeszenarien jeweils angepasste Lösungen. Bisherige Ansätze reichen von GNSS-gestützter (Global Navigation Satellite System) Positionierung (Patrik u. a. 2019) über markerbasierte visuelle Verfahren (Wubben u. a. 2019) bis hin zu markerfreien Systemen, die Landezonen eigenständig aus Sensordaten ableiten (Bartolomei u. a. 2022). Dabei zeigt sich eine wiederkehrende Einschränkung: Systeme zur Hindernisvermeidung navigieren erfolgreich durch komplexe Umgebungen, ohne gezielt zu landen (Loquercio u. a. 2021; Xu u. a. 2024), während Landesysteme überwiegend in hindernisfreien Szenarien operieren (Polvara u. a. 2020; Ladosz u. a. 2024). Die Kombination beider Teilaufgaben, Präzisionslandung mit gleichzeitiger Hindernisvermeidung, ist insbesondere in natürlichen Umgebungen mit dichter Vegetation bislang kaum untersucht. Weinberge sind ein konkretes Beispiel solcher Umgebungen: Reihenabstände von typischerweise 1,5 bis 2 m und dichtes Blattwerk erzeugen beengte Platzverhältnisse, die eine autonome Landung zwischen den Reihen erheblich erschweren.

Reinforcement Learning (RL) hat sich in jüngerer Zeit als wirksame Lösungsstrategie für Drohnenaufgaben etabliert (Kaufmann, Bauersfeld, Loquercio u. a. 2023; Hodge u. a. 2021;

Amendola u. a. 2024): Ein RL-Agent erlernt eine Steuerungsstrategie durch Interaktion mit einer simulierten Umgebung, ohne dass ein explizites Umgebungsmodell oder manuell definierte Steuerungsregeln benötigt werden. In Kombination mit Convolutional Neural Networks (CNNs) können aufgabenrelevante Merkmale direkt aus Sensordaten extrahiert werden (Mnih u. a. 2015), was eine bildbasierte Hindernisvermeidung ohne manuelle Merkmalsdefinition ermöglicht.

Drohnen unterliegen engen Gewichts- und Energiebeschränkungen, die die verfügbare Sensorausstattung limitieren; gleichzeitig können lernbasierte Ansätze begrenzte Sensorik algorithmisch kompensieren (Semerikov u. a. 2025). Die vorliegende Arbeit nutzt dieses Prinzip und untersucht, ob ein RL-Agent mit lediglich einer Tiefenkamera und einem propriozeptiven Zustandsvektor in Simulation lernen kann, Zielnavigation und Hindernisvermeidung in hindernisreichen Umgebungen zu vereinen.

1.2 Zielstellung

Ziel der Arbeit ist die Entwicklung einer Reinforcement-Learning-Pipeline, die eine autonome Multirotor-Drohne befähigt, mithilfe lokaler Tiefensensorik in einer hindernisreichen Umgebung sicher zu landen. Das System kombiniert einen propriozeptiven Zustandsvektor mit tiefenbildbasierter Umgebungswahrnehmung, um Hindernisse während des Landevorgangs zu erkennen und ihnen auszuweichen. Die Leistungsfähigkeit des trainierten Agenten wird anhand einer abstrakten Trainingsumgebung evaluiert und anschließend im Zero-Shot-Transfer auf eine Umgebung mit realistischer Vegetationsstruktur getestet.

Dadurch ergeben sich folgende Fragestellungen:

Hauptfragestellung

Inwiefern kann ein Reinforcement-Learning-basierter Ansatz eine autonome Drohne befähigen, mithilfe lokaler Tiefensensorik in hindernisreichen Umgebungen sicher zu landen?

Nebenfragestellungen

1. Wie leistungsfähig ist der trainierte Agent in der Trainingsumgebung, und welche Faktoren begrenzen die Erfolgsrate?
2. In welchem Maß lässt sich die erlernte Navigationsleistung auf eine geometrisch verschiedenartige Umgebung mit realistischer Vegetationsstruktur übertragen?
3. Welche Leistungslücke besteht zwischen einem lernbasierten Agenten mit lokaler Sensorik und einem Pfadplaner mit vollständiger Umgebungsinformation?

1.3 Aufbau der Arbeit

(TODO: BESCHREIBUNG VON KAPITEL 2 ANPASSEN, SOBALD DAS KAPITEL ÜBERARBEITET IST.) Kapitel 2 führt die für das Verständnis der Arbeit notwendigen Grundlagen des Reinforcement Learning ein, von der Formalisierung als Markov-Entscheidungsprozess bis zum verwendeten PPO-Algorithmus. Kapitel 3 ordnet die Arbeit in den Forschungsstand ein und identifiziert die adressierte Forschungslücke anhand bestehender Arbeiten zu autonomer Drohnenlandung, tiefenbildbasierter Hindernisvermeidung und lernbasierter Drohnensteuerung. Kapitel 4 beschreibt das entwickelte System: die hierarchische Steuerungsarchitektur, die MDP-Formulierung, die CNN-MLP-Netzwerkarchitektur, die Trainingsumgebung sowie das zweiphasige Evaluationsprotokoll. Kapitel 5 präsentiert die experimentellen Ergebnisse des Trainings und der zweiphasigen Evaluation sowie den Vergleich mit einem klassischen Pfadplaner. Kapitel 6 interpretiert die Ergebnisse im Kontext der Forschungsfragen und diskutiert Limitationen. Kapitel 7 beantwortet die Forschungsfragen zusammenfassend und skizziert Perspektiven für zukünftige Forschung.

2 Fachliche Grundlagen

Dieses Kapitel führt die theoretischen Grundlagen ein, auf denen das in dieser Arbeit entwickelte System aufbaut. Abschnitt 2.1 behandelt Reinforcement Learning als Lernparadigma und formalisiert das Problem als Markov Decision Process. Abschnitt 2.2 stellt Lösungsverfahren vor, ausgehend von tabellarischen Methoden über Funktionsapproximation bis hin zu Proximal Policy Optimization, dem in dieser Arbeit eingesetzten Trainingsalgorithmus. Abschnitt 2.3 beschreibt Convolutional Neural Networks als Architektur zur Verarbeitung räumlicher Sensordaten. Abschnitt 2.4 definiert achsenparallele Begrenzungsboxen, die in dieser Arbeit zur Kollisionserkennung verwendet werden.

2.1 Reinforcement Learning

Reinforcement Learning bezeichnet das Lernen aus Interaktionen mit einer Umgebung, um ein Ziel zu erreichen. Ein Agent wählt in aufeinanderfolgenden Situationen Aktionen und erhält dafür numerische Belohnungen. Aus diesen Erfahrungen muss er eigenständig ableiten, welche Aktionen nicht nur unmittelbar, sondern auch langfristig den höchsten Ertrag erzielen. Sofern nicht anders angegeben, folgt die Darstellung in diesem Abschnitt Sutton und Barto (2020).

2.1.1 Markov Decision Process

Ein *Markov Decision Process* (MDP) ist ein Modell, mit dem Reinforcement Learning Probleme präzise formalisiert werden können. Der Lerner und Entscheider wird als **Agent** bezeichnet; alles außerhalb, womit der Agent interagiert und was er nicht willkürlich verändern kann, bildet die **Umgebung**.

Die Interaktion zwischen Agent und Umgebung erfolgt in diskreten Zeitschritten $t = 0, 1, 2, \dots$. In jedem Schritt beobachtet der Agent einen **Zustand** $S_t \in \mathcal{S}$, wählt auf dessen Grundlage eine **Aktion** $A_t \in \mathcal{A}$ und erhält einen Zeitschritt später eine numerische **Belohnung** $R_{t+1} \in \mathcal{R}$ sowie einen neuen Zustand S_{t+1} (Abbildung 2.1).

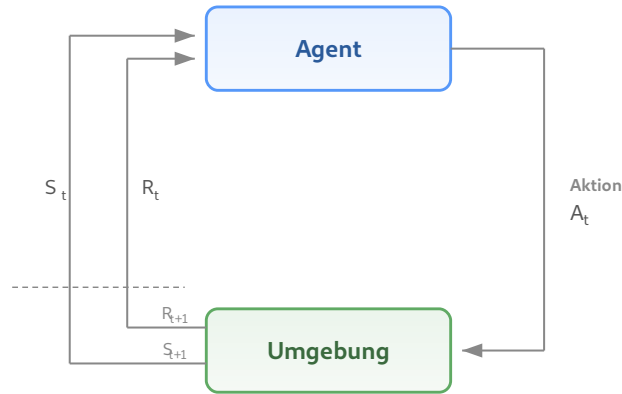


Abbildung 2.1.: Interaktion zwischen Agent und Umgebung in einem MDP (nach Sutton und Barto 2020, S. 48).

Die Übergangsdynamik der Umgebung wird durch die Funktion

$$p(s', r \mid s, a) = \Pr\{S_{t+1} = s', R_{t+1} = r \mid S_t = s, A_t = a\} \quad (2.1)$$

vollständig charakterisiert, welche die Wahrscheinlichkeit angibt, ausgehend vom Zustand s bei gewählter Aktion a in den Folgezustand s' überzugehen und dabei die Belohnung r zu erhalten. Wesentlich ist dabei die *Markov-Eigenschaft*: Folgezustand und Belohnung hängen ausschließlich vom gegenwärtigen Zustand-Aktions-Paar (s, a) ab.

Das Ziel des Agenten besteht darin, durch Interaktion mit der Umgebung eine Strategie $\pi(a \mid s)$ zu identifizieren, welche die über die Zeit kumulierte Belohnung maximiert. Eine solche Strategie wird als **optimale Strategie** bezeichnet und mit π_* gekennzeichnet.

2.2 Lösungen von MDPs

Welche Verfahren zur Bestimmung einer optimalen Strategie geeignet sind, hängt maßgeblich von der Formulierung des Reinforcement-Learning-Problems ab. Die klassische Theorie liefert exakte Lösungen ausschließlich für endliche MDPs mit vollständig bekanntem Übergangsmodell; ihre Grundideen lassen sich jedoch auf komplexere Problemklassen verallgemeinern. Dieser Abschnitt behandelt die Lösung von MDPs. Eingeführt werden zunächst tabellarische Verfahren, anschließend Funktionsapproximation und schließlich Policy-Gradient-Methoden, die in das Verfahren der *Proximal Policy Optimization* (PPO) münden.

2.2.1 Dynamische Programmierung

Das Ziel des Agenten besteht in der Maximierung des erwarteten Ertrags $G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$ mit Diskontierungsfaktor $\gamma \in [0, 1)$. Die Güte einer Strategie π wird durch die Zustands-

wertfunktion $v_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]$ charakterisiert, die den erwarteten Ertrag ausgehend von einem Zustand angibt. Die Zustandswertfunktion der optimalen Strategie π_* wird als v_* bezeichnet. Ihre direkte Berechnung setzt jedoch ein vollständig bekanntes Übergangsmodell und einen abzählbaren Zustandsraum voraus. In der Praxis sind beide Voraussetzungen selten gleichzeitig erfüllt: Das Übergangsmodell physikalischer Systeme ist in der Regel nicht vollständig bekannt, und selbst bei bekanntem Modell übersteigt der Zustandsraum realistischer Probleme typischerweise jene Dimensionen, in denen eine exakte Lösung mit vertretbarem Rechenaufwand möglich wäre.

2.2.2 Monte-Carlo- und Temporal-Difference-Verfahren

Das fehlende Modell als erste Einschränkung wird durch *sample-basierte* Verfahren wie Monte-Carlo- und Temporal-Difference-Methoden (TD) aufgehoben. Monte-Carlo-Methoden benötigen den vollständigen Episodenenertrag G_t und können die Wertschätzung daher erst nach Abschluss einer Episode aktualisieren. TD-Verfahren hingegen nutzen *Bootstrapping* und korrigieren ihre Schätzung nach jedem einzelnen Zeitschritt anhand der geschätzten Bewertung des Folgezustands:

$$V(S_t) \leftarrow V(S_t) + \alpha [R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]. \quad (2.2)$$

TD-Verfahren bilden die Grundlage der meisten modernen Reinforcement-Learning-Algorithmen: Sie können online angewandt werden, benötigen wenig Rechenaufwand und lassen sich mit Funktionsapproximation auf große Zustandsräume skalieren.

2.2.3 Funktionsapproximation

Die Dimensionalität des Zustandsraums als zweite Einschränkung kann durch **Funktionsapproximation** überwunden werden. Statt jeden Zustand mit einem eigenen Eintrag zu versehen, wird die Wertfunktion durch eine parametrisierte Funktion $\hat{v}(s, \mathbf{w})$ mit $\mathbf{w} \in \mathbb{R}^d$ und $d \ll |\mathcal{S}|$ dargestellt und in Richtung v_π optimiert. Parametrisierte Funktionen erlauben Generalisierung: Erfahrungen, die über einen Zustand gemacht wurden, lassen sich auf benachbarte Zustände übertragen; gleichzeitig werden kontinuierliche Zustandsräume handhabbar. Als Parametrisierung eignen sich insbesondere neuronale Netze, da diese als universelle Funktionsapproximatoren in der Lage sind, beliebige stetige Funktionen auf kompakten Teilmengen beliebig genau anzunähern (Hornik u. a. 1989).

Während die Funktionsapproximation das Problem kontinuierlicher Zustandsräume löst, bleibt ein zweites Hindernis bei kontinuierlichen *Aktionsräumen* bestehen. Wertbasierte Verfahren wählen Aktionen gemäß $a = \arg \max_{a'} \hat{q}(s, a', \mathbf{w})$; dieses Maximum ist in kontinuierlichen Aktionsräumen jedoch im Allgemeinen nicht effizient berechenbar.

2.2.4 Policy-Gradient-Methoden

Eine Alternative ist, die Strategie nicht über eine Wert-Funktion abzuleiten, sondern direkt als parametrisierte Funktion $\pi(a \mid s, \theta)$ zu beschreiben und ihre Parameter θ durch Gradientenaufstieg zu optimieren. Der entscheidende Vorteil für diese Arbeit: Policy-Gradient-Methoden können auf natürliche Weise kontinuierliche Aktionsräume handhaben, wie sie in der Drohnensteuerung auftreten.

In der Praxis wird der Policy-Gradient-Ansatz häufig als *Actor-Critic-Architektur* umgesetzt: Ein *Actor*-Netz repräsentiert die Strategie $\pi(\cdot \mid \cdot, \theta)$, ein *Critic*-Netz schätzt eine Wert-Funktion $\hat{v}(\cdot, \mathbf{w})$. Die Wertschätzung des Critics dient als Referenzniveau, das die Varianz der Gradientenschätzung reduziert, ohne sie zu verzerren.

2.2.5 Proximal Policy Optimization

Eine moderne und in der Praxis weit verbreitete Variante des Policy-Gradient-Ansatzes ist *Proximal Policy Optimization* (PPO) (Schulman, Wolski u. a. 2017), ein Actor-Critic-Verfahren, das die Trainingsstabilität von Trust-Region-Methoden erreicht und dabei wesentlich einfacher zu implementieren ist.

Zu große Strategieänderungen pro Optimierungsschritt können Policy-Gradient-Verfahren destabilisieren, da die gesammelten Trajektorien für die veränderte Strategie nicht mehr repräsentativ sind (Schulman, Wolski u. a. 2017). Trust-Region-Methoden wie TRPO (Schulman, Levine u. a. 2015) begrenzen daher die KL-Divergenz zwischen aufeinanderfolgenden Strategien, erfordern dafür jedoch Approximationen zweiter Ordnung. PPO erzielt eine vergleichbare Stabilisierung allein durch eine geklippte Zielfunktion:

$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_t \right) \right], \quad (2.3)$$

wobei $r_t(\theta) = \pi_\theta(a_t \mid s_t) / \pi_{\theta_{\text{alt}}}(a_t \mid s_t)$ das Wahrscheinlichkeitsverhältnis zwischen neuer und alter Strategie ist, $\hat{A}_t$ der Vorteilsschätzer und ε die Clipping-Grenze. Das Clipping bewirkt *pessimistische* Updates: Verbessert eine Strategieänderung das Objective, wird der Anreiz bei $r_t > 1 + \varepsilon$ gekappt; verschlechtert sie es, greift die Minimumbildung und die

Korrektur bleibt uneingeschränkt (Abbildung 2.2). So werden zu große Schritte in Richtung guter Erfahrungen gebremst, während schlechte Aktionen voll korrigiert werden.

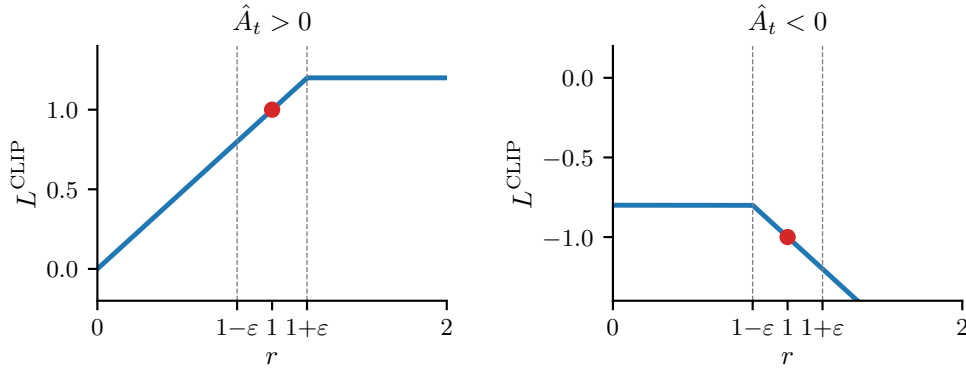


Abbildung 2.2.: L^{CLIP} als Funktion des Wahrscheinlichkeitsverhältnisses r . Links ($\hat{A}_t > 0$): der Anreiz wird bei $1 + \epsilon$ gekappt. Rechts ($\hat{A}_t < 0$): die Korrektur bleibt uneingeschränkt (nach (Schulman, Wolski u. a. 2017), Fig. 1).

Das vollständige PPO-Objective kombiniert das Clipping mit zwei weiteren Termen:

$$L(\theta) = L^{\text{CLIP}}(\theta) - c_1 L^{\text{VF}}(\theta) + c_2 H[\pi_\theta](s_t), \quad (2.4)$$

wobei $L^{\text{VF}} = (V_\theta(s_t) - V_t^{\text{targ}})^2$ der quadratische Fehler der Wertfunktionsschätzung des Critics ist und $H[\pi_\theta]$ einen Entropie-Bonus bezeichnet, der vorzeitige Konvergenz der Aktionsverteilung verhindert. Ohne diesen Term kann die Standardabweichung gegen null konvergieren (*Std-Collapse*), was die Strategie irreversibel auf eine deterministische Aktion festlegt.

Für die Schätzung des Vorteils $\hat{A}_t$ wird *Generalized Advantage Estimation* (GAE) (Schulman, Moritz u. a. 2016) eingesetzt, das über den Parameter λ einen Kompromiss zwischen Bias und Varianz der Vorteilsschätzung steuert.

2.3 Convolutional Neural Networks

Convolutional Neural Networks (CNNs) sind eine Klasse neuronaler Netze, die speziell für die Verarbeitung von Daten mit räumlicher Struktur entwickelt wurden, etwa Bilder in Form zweidimensionaler Pixelarrays. Sofern nicht anders angegeben, folgt die Darstellung in diesem Abschnitt LeCun u. a. (2015).

Die Architektur eines CNN besteht aus einer Kaskade von Faltungs- und Pooling-Schichten. In einer *Faltungsschicht* sind die Einheiten in sogenannten *Feature Maps* organisiert. Jede Einheit ist nur mit einem lokalen Ausschnitt der vorherigen Schicht verbunden und verrechnet diesen mit einem *Filter* (auch *Kernel*), einer kleinen quadratischen Gewichtsmatrix (Botsch 2023). Dabei werden die überdeckten Eingabewerte elementweise mit den Filtergewichten

multipliziert und zu einem einzelnen Ausgabewert aufsummiert. Alle Einheiten innerhalb einer Feature Map teilen denselben Filter; mathematisch entspricht diese Operation einer diskreten Faltung. Verschiedene Feature Maps innerhalb einer Schicht verwenden unterschiedliche Filter und erkennen dadurch unterschiedliche lokale Muster. Die Filtergewichte werden nicht manuell festgelegt, sondern während des Trainings gelernt (Botsch 2023). Die gemeinsame Nutzung der Gewichte beruht auf zwei Eigenschaften natürlicher Bilddaten: Benachbarte Werte bilden häufig korrelierte lokale Muster, und diese Muster treten unabhängig von ihrer Position im Bild auf.

Pooling-Schichten fassen die Aktivierungen benachbarter Einheiten zusammen, typischerweise durch Auswahl des Maximums innerhalb eines lokalen Fensters (*Max-Pooling*) (Botsch 2023). Dies reduziert die räumliche Auflösung der Repräsentation und erzeugt eine gewisse Invarianz gegenüber kleinen Verschiebungen im Eingabebild.

Durch das Stapeln mehrerer Faltungs- und Pooling-Stufen entsteht eine hierarchische Merkmalsextraktion: Frühe Schichten erkennen einfache lokale Strukturen wie Kanten, mittlere Schichten kombinieren diese zu komplexeren Motiven, und tiefe Schichten repräsentieren ganze Objektteile. Die resultierende kompakte Repräsentation kann als Eingabe für nachgeschaltete Netzwerkkomponenten dienen. Mnih u. a. (2015) zeigten erstmals, dass ein CNN in dieser Rolle als Merkmalsextraktor innerhalb eines Reinforcement-Learning-Systems eingesetzt werden kann, indem sie eine Strategie direkt aus hochdimensionalen Bilddaten lernten.

2.4 Achsenparallele Begrenzungsboxen

Eine **achsenparallele Begrenzungsbox** (*Axis-Aligned Bounding Box*, AABB) ist das kleinste achsenparallele Quader, das ein gegebenes Objekt vollständig umschließt. Sie wird durch zwei Punkte definiert: das komponentenweise Minimum und Maximum aller Objektpunkte,

$$\mathbf{p}_{\min} = (\min_i x_i, \min_i y_i, \min_i z_i), \quad \mathbf{p}_{\max} = (\max_i x_i, \max_i y_i, \max_i z_i), \quad (2.5)$$

wobei (x_i, y_i, z_i) die Eckpunkte bzw. Oberflächenpunkte des Objekts bezeichnen.

Die AABB besitzt eine *konservative* Eigenschaft: Da sie das Objekt vollständig umschließt, wird jede tatsächliche Kollision mit dem Objekt auch als Kollision mit der AABB erkannt. Im Umkehrschluss kann ein Punkt jedoch innerhalb der AABB liegen, ohne das eingeschlossene Objekt zu berühren. Der Approximationsfehler wächst mit der Abweichung der Objektgeo-

metrie von einer quaderförmigen Gestalt; bei konkaven oder stark unregelmäßigen Formen kann die AABB ein erheblich größeres Volumen als das Objekt selbst einnehmen.

Für die Kollisionserkennung in Echtzeitsystemen bietet die AABB einen günstigen Kompromiss: Der Abstandstest reduziert sich auf sechs skalare Vergleiche pro Achse und ist damit wesentlich effizienter als Distanzberechnungen auf der tatsächlichen Objektgeometrie. Dem steht die beschriebene konservative Approximation gegenüber, die zu falsch-positiven Kollisionserkennungen führen kann.

3 Stand der Forschung

3.1 Autonomes Landen von Drohnen

Bisherige Arbeiten zum autonomen Landen von Drohnen lassen sich in zwei grundlegende Kategorien einteilen: traditionelle und bildbasierte Methoden. Traditionelle Methoden stützen sich auf Signale externer Infrastruktur wie GNSS-Satelliten oder Funkbaken, um Position und Lage der Drohne zu bestimmen. Bildbasierte Methoden hingegen nutzen Kameradaten zur Wahrnehmung der Umgebung und lassen sich weiter unterteilen in markerbasierte Systeme, die definierte visuelle Referenzpunkte zur Lokalisierung verwenden, und markerfreie Systeme, die adäquate Landezonen eigenständig aus Bilddaten ableiten und somit keine zusätzliche Bodeninfrastruktur erfordern. Diese Einteilung orientiert sich an der Taxonomie von Arafat u. a. (2023).

Orthogonal zur Sensorik lassen sich Ansätze danach unterscheiden, welche Teilaufgaben des Guidance, Navigation and Control (GNC) (Kanellakis und Nikolakopoulos 2017) sie adressieren und wie diese zusammenwirken. Modulare Methoden zerlegen das Problem in getrennte Komponenten wie Obstacle Detection, Pose-Estimation oder Path-Planning, die als aufeinanderfolgende Stufen einer Pipeline gelöst werden (Kanellakis und Nikolakopoulos 2017; Wang u. a. 2024). End-to-End-Methoden hingegen bilden Sensorinput direkt auf Steuerkommandos ab, ohne solche expliziten Zwischenstufen (Wang u. a. 2024).

GNSS ist die verbreitetste Methode zur Positionsbestimmung von Drohnen: Ein einzelnes GNSS-Modul liefert eine Genauigkeit von 5–10 m und bildet die Grundlage für Missionsplanung und -ausführung (Jarraya u. a. 2025). Dass autonomes Landen bei bekannter Relativposition grundsätzlich beherrschbar ist, zeigen Voos und Bou-Ammar (2010) mittels nichtlinearer Regelung auf stationären und beweglichen Zielen. Die zentrale Herausforderung liegt somit nicht im Regelungsentwurf, sondern in der Genauigkeit der zugrunde liegenden Positionsdaten.

In der Praxis erreicht GNSS-basierte Landung jedoch nur Meter-Genauigkeit. Patrik u. a. (2019) berichten von einer durchschnittlichen Landeabweichung von 1,11 m; auch beim Open-Source-Autopiloten PX4 beträgt die GNSS-basierte Landegenauigkeit mehrere Meter (Meier u. a. 2015; Wubben u. a. 2019), sodass für höhere Präzision zusätzliche Sensoren erforderlich sind (*Precision Landing* 2026). Die Positionsgenauigkeit lässt sich zwar durch RTK-Korrekturen auf Zentimeter-Niveau steigern — Tavasci u. a. (2024) berichten

zentimetergenaue Echtzeit-Navigation unter idealen Bedingungen —, allerdings fällt die Genauigkeit in schwierigen GNSS-Umgebungen, etwa bei Signalabschattung durch Vegetation oder Gebäude, auf Meter-Niveau zurück. Gerade in Agrarumgebungen mit Baumkronen und Laubdächern sind solche Bedingungen typisch.

GNSS bleibt damit als robuste Grobpositionierung unverzichtbar, weist jedoch für Präzisionslandung in strukturierten Umgebungen fundamentale Limitationen auf: Die Genauigkeit reicht nicht für das Landen zwischen Baumreihen oder Weinreben mit einem typischen Reihenabstand von 1,5–2 m. Zudem liefert GNSS keinerlei Umgebungswahrnehmung; Veränderungen der Umgebung, Hindernisse oder beengte Platzverhältnisse bleiben unerkannt.

Bildbasierte Landesysteme mit visuellen Markern bieten eine deutlich höhere Präzision als rein GNSS-gestützte Methoden. So erreicht das ArUco-basierte System von Wubben u. a. (2019) durch deterministische Bildverarbeitung eine durchschnittliche Landeabweichung von 11 cm bei zuverlässiger Markerdetektion aus bis zu 30 m Höhe. Solche klassischen Ansätze setzen jedoch voraus, dass die Markererkennung unter allen Betriebsbedingungen robust funktioniert — eine Annahme, die bei Verdeckung, wechselnden Lichtverhältnissen oder beschädigten Markern nicht gegeben ist.

Reinforcement Learning bietet hier einen konzeptionellen Vorteil: Statt auf handspezifizierte Detektionsalgorithmen angewiesen zu sein, lernt ein RL-Agent eine End-to-End-Steuerung direkt aus Bilddaten und kann dabei implizit Robustheit gegenüber visuellen Störungen erwerben. Polvara u. a. (2020) demonstrieren diesen Ansatz erstmals für die autonome Landung: Ihr System nutzt sequentielle Deep-Q-Networks mit diskretem Aktionsraum und einer Divide-and-Conquer-Strategie, die den Landeprozess in Marker-Ausrichtung und vertikalen Abstieg unterteilt — ein Ansatz, der das Problem dünn besetzter Belohnungssignale adressiert, indem die komplexe Aufgabe in einfachere Teilziele zerlegt wird. Als einziger Sensor dient eine niedrig aufgelöste monokulare Graustufenkamera. Ausschließlich in Simulation mit Domain Randomization trainiert, erzielt das System Erfolgsraten von 89 % für den vertikalen Abstieg, die im realen Flug auf 61 % sinken. Der Robustheitsvorteil gegenüber klassischen Methoden zeigt sich unter erschwerten Bedingungen: Bei verrauschten Markertexturen erreicht das RL-System noch 51 %, während ein konventioneller AR-Tracker vollständig versagt — allerdings ist auch dieser Wert für sicherheitskritische Anwendungen unzureichend. Eine zentrale methodische Einschränkung bleibt der diskrete Aktionsraum, der die Steuerungspräzision limitiert.

Ladosz u. a. (2024) erweitern den Ansatz auf bewegliche Landeplattformen und adressieren diese Einschränkung durch einen kontinuierlichen Aktionsraum, der erstmals auch die Vertikalgeschwindigkeit einschließt — bei früheren Arbeiten wird der vertikale Abstieg typischerweise regelbasiert gesteuert. Der Agent erhält neben Graustufenbildern auch Daten einer inertialen Messeinheit (IMU; Geschwindigkeit und Lage) als Zustandsinformation. In der Simulation erreicht das System Erfolgsraten von bis zu 97,4 % bei einer Landeabweichung von 0,52 m; im realen Transfer sinkt die Rate auf rund 70 %. Die Autoren zeigen zudem, dass Domain Randomization zwar notwendig für den Sim-to-Real-Transfer ist, eine zu starke Randomisierung die Performance jedoch signifikant verschlechtert. Die Experimente beschränken sich auf einfache Szenarien ohne Hindernisse und gleichmäßig beleuchtete Innenräume.

Trotz dieser Fortschritte weisen markerbasierte Systeme grundlegende Einschränkungen auf, die unabhängig von der gewählten Steuerungsmethode gelten: Sie setzen vorplatzierte visuelle Infrastruktur am Landeziel voraus, was ihren Einsatz auf vordefinierte Orte begrenzt und in unbekannten Umgebungen unpraktikabel macht. Hinzu kommen methodische Limitationen der RL-basierten Ansätze: Beide Systeme wurden in vereinfachten Simulationsumgebungen ohne Hindernisse trainiert, und der Transfer in die reale Welt geht mit deutlichen Leistungseinbußen einher — Ladosz u. a. (2024) berichten einen Rückgang der Erfolgsrate von 97,4 % auf rund 70 % (vgl. auch Polvara u. a. 2020). Diese Limitationen motivieren die Entwicklung markerfreier Systeme, die Landezonen eigenständig aus Bilddaten ableiten und keine zusätzliche Bodeninfrastruktur erfordern.

Markerfreie Verfahren ermöglichen den Einsatz in unbekannten, unstrukturierten und dynamischen Umgebungen, da sie keine manuelle Vorbereitung der Landezone erfordern — eine Eigenschaft, die insbesondere für Notlandungen oder den Betrieb in unzugänglichen Gebieten entscheidend ist (Yang u. a. 2018; Bartolomei u. a. 2022). Die Bandbreite der Ansätze reicht dabei von klassischen modularen Pipelines bis hin zu End-to-End-Steuerungen mittels Deep Reinforcement Learning.

Yang u. a. (2018) verfolgen einen modularen Ansatz, bei dem aus monokularer SLAM-Rekonstruktion eine 3D-Punktwolke erzeugt wird, aus der anhand von Oberflächensemantik und Geländehöhe sichere Landezonen in einer Grid-Map identifiziert werden. Das System nutzt ausschließlich Visual-Inertial-Odometrie zur Zustandsschätzung und benötigt weder GNSS noch externe Lokalisierungsinfrastruktur. Es demonstriert Landungen in verschiede-

nen Umgebungen, weist jedoch aufgrund der hohen Pipeline-Komplexität eine Landezeit von rund zwei Minuten auf.

Einen End-to-End-Ansatz wählen Bartolomei u. a. (2022): Ihr RL-Agent bildet Tiefenbilder und semantische Segmentierungen — sogenannte Mid-Level-Repräsentationen, die generischer als Rohbilder sind und schnelleres Training ermöglichen — direkt auf diskrete Steuerkommandos ab. Das System erreicht in der Simulation Erfolgsraten von über 70 % bei minimaler Sensorausstattung mit lediglich einer monokularen RGB-Kamera. Ein methodisch relevanter Aspekt ist die Sim-to-Real-Strategie: Die Policy wird zunächst mit Ground-Truth-Tiefen- und Semantikdaten trainiert und anschließend mit verrauschten Prädiktionen neuronaler Netze feinabgestimmt, um den Simulationstransfer zu ermöglichen. Im Realflug landete die Policy erfolgreich, während sowohl eine kommerzielle als auch eine State-of-the-Art-Lösung am verrauschten Sensorinput scheiterten. Einschränkend setzt das System voraus, dass semantische Segmentierungen und Tiefenbilder stets verfügbar sind, und der diskrete Aktionsraum (zehn mögliche Aktionen) limitiert die Steuerungsflexibilität.

Dass Deep Reinforcement Learning auch unter dynamischen Umgebungsbedingungen effektive Landesteuerungen ermöglicht, zeigen Peter u. a. (2024) mit TornadoDrone: Das bioinspirierte DRL-Framework übertrifft eine konventionelle PID-Regler-Baseline (Proportional-Integral-Derivative) bei Landungen unter starken Windturbulenzen deutlich, stützt sich in den realen Experimenten jedoch auf ein Vicon-Lokalisierungssystem, das unter Einsatzbedingungen nicht verfügbar ist.

3.2 Tiefenbildbasierte Hindernisvermeidung

Loquercio u. a. (2021) demonstrieren ein End-to-End-Navigationssystem für Hochgeschwindigkeitsflüge durch komplexe, unbekannte Umgebungen mit ausschließlich bordeigener Sensorik und Rechenleistung. Das System nutzt Tiefenbilder einer Stereokamera und IMU-Daten, um kollisionsfreie Kurzzeittrajektorien zu erzeugen, die von einem nachgeschalteten PID-Regler ausgeführt werden. Trainiert wird ausschließlich in Simulation mittels Privileged Learning, einer Imitation eines Experten mit Zugang zu perfekter Zustandsinformation, wobei lediglich einfache Hindernisgeometrien (Zylinder, Quader, Baumstämme) verwendet werden. Durch die Simulation realistischer Sensorrauschens gelingt ein Zero-Shot-Transfer in reale Umgebungen, die während des Trainings nie erfahren wurden: dichte Wälder, verschneites Gelände und eingestürzte Gebäude. Bei niedrigen Geschwindigkeiten erreicht das System eine Erfolgsrate von 100 % in allen getesteten Umgebungen; im Vergleich zu modu-

laren State-of-the-Art-Planern reduziert der Ansatz die Ausfallrate um den Faktor 10. Eine Einschränkung ist die geringe Positioniergenauigkeit, da Erfolg bereits bei einem Abstand von 5 m zum Ziel definiert wird und der Fokus auf Geschwindigkeit statt Präzision liegt. Methodisch relevant für die vorliegende Arbeit ist der Nachweis, dass ein lernbasierter Ansatz mit Tiefenbildern und hierarchischer Architektur (Policy $\rightarrow$ PID) in dicht bewachsenen natürlichen Umgebungen robust navigieren kann. Die vorliegende Arbeit verfolgt eine vergleichbare Architektur, ersetzt jedoch Privileged Learning durch Reinforcement Learning und verschiebt den Fokus von Hochgeschwindigkeitsnavigation auf Präzisionslandung in beengten Umgebungen.

Kim u. a. (2022) verfolgen einen zweistufigen Ansatz, bei dem zunächst ein vortrainiertes CNN Tiefenbilder aus monokularen RGB-Aufnahmen schätzt und anschließend ein Dueling-Double-DQN auf Basis dieser geschätzten Tiefeninformation diskrete Ausweichentscheidungen trifft. Als einziger externer Sensor dient die bordeigene Monokularkamera der Drohne; Zustandsinformation wie Position oder Geschwindigkeit wird nicht benötigt. Das System wird in Simulation trainiert und ohne weitere Anpassung auf eine reale Drohne übertragen, wo es enge Korridore mit texturlosen Wänden, Personen und Kisten kollisionsfrei durchfliegt. Evaluert wird anhand der zurückgelegten Strecke ohne Kollision; eine gezielte Navigation zu einem definierten Ziel ist nicht Teil des Systems. Die Testumgebungen beschränken sich auf strukturierte Innenräume; Experimente in natürlichen oder vegetationsreichen Umgebungen werden nicht durchgeführt. Der Ansatz demonstriert, dass selbst aus monokularer RGB-Eingabe geschätzte Tiefeninformation für lernbasierte Hindernisvermeidung ausreicht, allerdings mit den Einschränkungen eines diskreten Aktionsraums und der Abhängigkeit von der Qualität der Tiefenschätzung.

Xu u. a. (2024) nutzen direkte Tiefenbilder, um mittels PPO eine kontinuierliche Policy für sichere Navigation in Umgebungen mit statischen und dynamischen Hindernissen zu trainieren. Die Architektur verarbeitet die Tiefenbilder zu einer 3D-Belegungskarte, die zusammen mit Drohnenzustandsdaten (Position, Geschwindigkeit, Orientierung) als Eingabe für das Actor-Critic-Netzwerk dient. Das Training erfolgt parallelisiert in Simulation mit Curriculum Learning, bei dem die Anzahl dynamischer Hindernisse schrittweise erhöht wird. Ein nachgeschalteter Safety Shield auf Basis von Velocity Obstacles ergänzt die gelernte Policy zur Laufzeit. Das System erreicht Zero-Shot-Transfer auf eine reale Drohne und erzielt in Szenarien mit dynamischen Hindernissen die geringste Kollisionsrate im Vergleich zu

bestehenden Methoden. Auch hier beschränken sich die Testumgebungen auf strukturierte Indoor-Szenarien mit geometrisch einfachen Hindernissen.

Gemeinsam zeigen diese Arbeiten, dass tiefenbildbasierte lernende Systeme Hindernisse zuverlässig vermeiden können, unabhängig davon ob die Tiefeninformation direkt gemessen (Loquercio u. a. 2021; Xu u. a. 2024) oder aus RGB-Bildern geschätzt wird (Kim u. a. 2022). Allerdings testen alle genannten Systeme in strukturierten Innenräumen oder Waldumgebungen; Evaluationen in landwirtschaftlichen Umgebungen mit Rebzeilen, niedrigen Baumkronen oder dichter Bodenvegetation fehlen.

Vereinzelte Arbeiten adressieren die Navigation in landwirtschaftlichen Umgebungen gezielt. Wei u. a. (2025) demonstrieren ein lernbasiertes Drohnensystem für die autonome Navigation durch Obstplantagen-Reihen: Ein VAE-Controller, trainiert mittels Imitation Learning aus Expertendemonstrationen, steuert einen Quadrotor auf Basis einer front-montierten RGB-Kamera kollisionsfrei zwischen Baumreihen. Das System generalisiert auf ungesehene Umgebungen und kommt ohne GPS aus, beschränkt sich jedoch auf horizontale Reihennavigation. Für Weinberge zeigen Martini u. a. (2022), dass ein DRL-Agent auf Basis von Tiefenbildern ohne Positionsinformation durch simulierte Rebzeilen navigieren kann, allerdings mit einem Bodenroboter statt einer Drohne. Beide Arbeiten bestätigen, dass Vegetation in landwirtschaftlichen Umgebungen eine eigenständige Navigationsherausforderung darstellt, die durch lernbasierte Ansätze mit minimaler Sensorik adressiert werden kann. Keine der Arbeiten behandelt jedoch die vertikale Landung zwischen Vegetationsstrukturen, die eine kombinierte Hindernisvermeidung und Präzisionspositionierung erfordert.

3.3 Reinforcement Learning für Drohnensteuerung

Über die Wahl der Sensorik hinaus beeinflusst die Steuerungsarchitektur die Leistungsfähigkeit lernbasierter Drohnensysteme maßgeblich, insbesondere der Aktionsraum und die Abstraktionsebene der Regelung. Die in den Abschnitten 3.1 und 3.2 betrachteten Arbeiten lassen sich auch entlang dieser Dimension einordnen: End-to-End-Systeme wie (Polvara u. a. 2020; Kim u. a. 2022; Ladosz u. a. 2024) erzeugen Bewegungskommandos direkt aus Sensorinput, während hierarchische Ansätze wie (Loquercio u. a. 2021; Bartolomei u. a. 2022) die Policy-Ausgabe an einen nachgeschalteten Regler delegieren. Xu u. a. (2024) kombinieren beide Paradigmen, indem sie eine PPO-Policy mit einem regelbasierten Safety Shield ergänzen.

Die Wahl der Abstraktionsebene beeinflusst die Lernperformance substantiell: Eßer u. a. (o. D.) zeigen in einer systematischen Studie über verschiedene Roboterplattformen und Aufgaben, dass der optimale Aktionsraum stark aufgabenabhängig ist und unterschiedliche Abstraktionsebenen zu deutlich verschiedenem Explorationsverhalten und finaler Performance führen. Speziell für Quadrotoren vergleichen Kaufmann, Bauersfeld und Scaramuzza (2022) drei Abstraktionsebenen: Geschwindigkeitskommandos, kollektiven Schub mit Drehraten sowie direkte Einzelmotorschübe. Die mittlere Ebene (Schub und Drehraten) erweist sich als beste Balance zwischen Agilität und Transferrobustheit; die höchste Abstraktion (Geschwindigkeit) schränkt die Manövrierfähigkeit ein, während die niedrigste (Einzelmotorschub) zu latenzempfindlich für den Realtransfer ist.

Eng mit der Wahl des Aktionsraums verknüpft ist die Wahl des Trainingsalgorithmus. Als Policy-Gradient-Verfahren eignet sich PPO (vgl. Abschnitt 2.2.5) besonders für kontinuierliche Aktionsräume und wird in der Drohnensteuerung häufig eingesetzt. Da Reinforcement Learning grundsätzlich große Datenmengen benötigt, hat sich GPU-parallele Simulation als effektiver Ansatz etabliert: Rudin u. a. (2022) zeigen, dass massiv-parallele Simulation mit hunderten bis tausenden Umgebungsinstanzen die Trainingszeit drastisch reduziert und schnellere Konvergenz ermöglicht. PPO eignet sich als On-Policy-Verfahren für dieses Paradigma, da es die parallel erzeugten Daten direkt verarbeitet und keinen Replay Buffer benötigt. Obwohl Off-Policy-Verfahren wie SAC eine höhere Sample-Effizienz bieten, zeigen Tayar u. a. (2025), dass PPO bei sicherheitskritischen Präzisionsaufgaben in beengten Räumen zuverlässiger konvergiert als SAC. In den in diesem Kapitel betrachteten Arbeiten setzen Bartolomei u. a. (2022), Xu u. a. (2024) und Kaufmann, Bauersfeld und Scaramuzza (2022) PPO ein; Kaufmann, Bauersfeld, Loquercio u. a. (2023) demonstrieren den bislang eindrucksvollsten Einsatz, bei dem ein ausschließlich in Simulation trainiertes System drei menschliche Weltmeister im physischen Drohnenrennen schlägt.

Die vorliegende Arbeit folgt dem hierarchischen Paradigma und wählt eine noch höhere Abstraktionsebene als die von Kaufmann, Bauersfeld und Scaramuzza (2022) untersuchten: Der RL-Agent gibt eine Zielposition vor, die ein kaskadierender PID-Regler (Position $\rightarrow$ Geschwindigkeit $\rightarrow$ Lage $\rightarrow$ Drehzahl) in Motorkommandos umsetzt. Diese Wahl opfert bewusst Agilität zugunsten reduzierter Lernkomplexität, da die Aufgabe Präzisionslandung statt agilen Flugs erfordert. Die Ausgabe des Agenten bleibt interpretierbar, der PID-Regler kann unabhängig vom RL-Training auf die Zielplattform abgestimmt werden, und die Lernkomplexität reduziert sich auf die Frage, *wohin* die Drohne fliegen soll.

Die Wahl der Sensorausstattung ist ein weiterer zentraler Designparameter: Drohnen sind aufgrund ihrer Energie- und Gewichtsbeschränkungen typischerweise nur mit minimaler Sensorik ausgestattet, bestehend aus Kameras und Distanzsensoren (Bartolomei u. a. 2022). Semerikov u. a. (2025) zeigen in ihrer Übersichtsarbeit, dass Vision-Inertial-Kombinationen eine gute Balance zwischen Wahrnehmungsleistung und Systemkomplexität bieten, während leistungsfähigere Sensorsetups entsprechend größere Drohnen mit höherer Lade- und Batteriekapazität erfordern. Gleichzeitig ermöglichen moderne Algorithmen, insbesondere lernbasierte Ansätze, dass auch mit leichtgewichtiger Sensorik anspruchsvolle Aufgaben gelöst werden können, die bisher komplexere Hardware voraussetzten. Die in diesem Kapitel betrachteten Arbeiten bestätigen diesen Trend: Erfolgreiche Hindernisvermeidung und Navigation gelingen mit einer einzelnen Kamera (Kim u. a. 2022; Polvara u. a. 2020; Bartolomei u. a. 2022) oder einer Tiefenkamera mit IMU-Zustandsdaten (Loquercio u. a. 2021; Xu u. a. 2024). Die vorliegende Arbeit folgt diesem Prinzip minimaler Sensorik: Als Eingabe dienen eine nach unten gerichtete Tiefenkamera und ein Zustandsvektor (Position, Orientierung, Geschwindigkeit), wobei die algorithmische Komplexität des RL-Agenten die begrenzte Sensorausstattung kompensiert.

Tabelle 3.1 stellt die in diesem Kapitel betrachteten Arbeiten anhand ihrer Kernmerkmale gegenüber.

Tabelle 3.1.: Vergleich lernbasierter Systeme zur autonomen Drohnennavigation und -landung. Die Spalten *Hind.* und *Landung* kennzeichnen, ob das System aktive Hindernisvermeidung bzw. Präzisionslandung adressiert.

Arbeit	Sensorik	Methode	Akt.-raum	Hind.	Umgebung	Landung
Polvara u. a. (2020)	Mono RGB	DQN	diskret	–	Indoor	✓
Ladosz u. a. (2024)	Mono RGB, IMU	DrQv2	kont.	–	Indoor	✓
Yang u. a. (2018)	Mono RGB	SLAM	–	✓	Outdoor	✓
Bartolomei u. a. (2022)	Tiefe, Semantik	PPO	diskret	✓	Outdoor	✓
Peter u. a. (2024)	Vicon, IMU	TD3	kont.	–	Indoor	✓
Loquercio u. a. (2021)	Stereo Tiefe, IMU	IL	kont.	✓	Outdoor	–
Kim u. a. (2022)	Mono RGB, Tiefe	DQN	diskret	✓	Indoor	–
Xu u. a. (2024)	Tiefe, Zustand	PPO	kont.	✓	Indoor	–
Wei u. a. (2025)	Mono RGB	IL	kont.	✓	Agrar	–
Martini u. a. (2022) ^a	Tiefe	SAC	kont.	✓	Agrar	–
Eigene Arbeit ^b	Tiefe, Zustand	PPO	kont.	✓	Agrar	✓

^a Bodenroboter (UGV). ^b Training in abstrakter Korridorumgebung; Evaluation in simuliertem Weinberg (Sim-to-Sim).

Zusammenfassend zeigt der Stand der Forschung drei wiederkehrende Limitationen RL-basierter Landesysteme: Erstens operieren die meisten Ansätze mit diskreten Aktionsräumen (Polvara u. a. 2020; Bartolomei u. a. 2022) oder vereinfachen den vertikalen Abstieg auf regelbasierte Steuerung — lediglich Ladosz u. a. (2024) setzen einen kontinuierlichen Aktionsraum

mit Vertikalkontrolle ein, beschränken sich jedoch auf hindernisfreie Innenräume. Zweitens werden Hindernisvermeidung und Präzisionslandung überwiegend getrennt adressiert (vgl. Tabelle 3.1): Systeme zur Hindernisvermeidung navigieren durch komplexe Umgebungen, ohne gezielt zu landen (Loquercio u. a. 2021; Kim u. a. 2022; Xu u. a. 2024), während Landesysteme in der Regel in hindernisfreien Szenarien operieren (Polvara u. a. 2020; Ladosz u. a. 2024). Drittens bleibt der Sim-to-Real-Transfer eine offene Herausforderung: Die Erfolgsraten sinken systematisch beim Übergang in die reale Welt — von 89 % auf 61 % (Polvara u. a. 2020), von 97,4 % auf rund 70 % (Ladosz u. a. 2024) —, und reale Experimente stützen sich auf Hilfsmittel wie Vicon-Lokalisierung (Peter u. a. 2024) oder gezieltes Fine-Tuning (Bartolomei u. a. 2022), die unter Einsatzbedingungen nicht verfügbar sind.

Keine der betrachteten Arbeiten kombiniert Präzisionslandung, aktive Hindernisvermeidung, tiefenbildbasierte Wahrnehmung und einen kontinuierlichen Aktionsraum in einer Agrarumgebung (vgl. Tabelle 3.1). Die vorliegende Arbeit adressiert die ersten beiden Limitationen: Sie untersucht, ob ein RL-Agent mit kontinuierlichem Aktionsraum und Tiefenbildern in Simulation lernen kann, in beengten Umgebungen mit Hindernissen autonom zu landen. Ein vollständiger Sim-to-Real-Transfer liegt außerhalb des Rahmens dieser Arbeit. Um die Transferfähigkeit der gelernten Strategie dennoch zu untersuchen, wird ein Zero-Shot-Transfer in eine geometrisch verschiedenartige Simulationsumgebung (Sim-to-Sim) als vereinfachte Annäherung an die dritte Limitation eingesetzt.

4 Methodik

Dieses Kapitel beschreibt das entwickelte System zur autonomen Drohnenlandung mit Hindernisvermeidung. Die in Abschnitt 3.3 motivierte hierarchische Steuerungsarchitektur bildet den Ausgangspunkt: Ein RL-Agent trifft Navigationsentscheidungen auf hoher Abstraktionsebene, während ein kaskadierender PID-Regler diese in Motorkommandos umsetzt. Die Aufgabe wird als Markov-Entscheidungsprozess formuliert (vgl. Abschnitt 2.1.1), in dem der Agent aus Tiefenbildern und einem Zustandsvektor eine Zielposition ableitet. Dieses bewusst minimalistische Sensorset, bestehend aus einer einzelnen Tiefenkamera und Eigenlageschätzung, folgt dem in Abschnitt 3.3 identifizierten Trend, algorithmische Komplexität anstelle zusätzlicher Hardware einzusetzen.

Training und Evaluation finden vollständig in Simulation statt. Der GPU-parallelisierte (Graphics Processing Unit) Physiksimulator Genesis erzeugt die für PPO benötigten großen Rollout-Batches aus bis zu 512 gleichzeitigen Umgebungsinstanzen. Ein Sim-to-Real-Transfer liegt außerhalb des Rahmens dieser Arbeit.

Abschnitt 4.1 gibt zunächst einen Überblick über die Systemarchitektur. Anschließend formalisiert Abschnitt 4.2 die Aufgabe als MDP mit Zustands-, Aktionsraum, Belohnungsfunktion und Terminierungsbedingungen. Abschnitt 4.3 beschreibt die CNN-MLP-Netzwerkarchitektur, Abschnitt 4.4 den PPO-Algorithmus einschließlich der Tanh-Aktionsverteilung. Die physische Trainingsumgebung wird in Abschnitt 4.5 vorgestellt, gefolgt vom PID-Regler (Abschnitt 4.6), dem Trainingsprozess (Abschnitt 4.7) und dem Evaluationsprotokoll (Abschnitt 4.8).

4.1 Systemüberblick

Das System gliedert sich in zwei Steuerungsebenen (Abbildung 4.1): Auf der oberen Ebene trifft ein RL-Agent alle 3,0 s eine Navigationsentscheidung. Auf der unteren Ebene setzt ein kaskadierender PID-Regler diese Entscheidung mit 100 Hz in Motordrehzahlen um und stabilisiert die Drohne zwischen zwei Entscheidungszeitpunkten. Das vergleichsweise lange Entscheidungsintervall ist eine bewusste Designentscheidung: Es stellt sicher, dass die Drohne zwischen zwei Entscheidungen zur Ruhe kommt und das Tiefenbild aus einer stabilen Fluglage aufgenommen wird, verhindert, dass der PID-Regler durch zu häufig wechselnde Sollwerte destabilisiert wird, und reduziert die Lernkomplexität, da der Agent weniger Entscheidungen pro Episode treffen muss. Empirisch bestätigte sich diese Wahl: Bei kürzeren Intervallen

(1,0 s und darunter) war die Drohne zwischen Entscheidungen nicht vollständig stabilisiert, was zu deutlich langsamerer Konvergenz oder ausbleibendem Lernerfolg führte.

Wahrnehmung. Der Agent erhält zwei Eingaben: einen normalisierten Zustandsvektor $\mathbf{o}_t \in \mathbb{R}^{17}$, der die relative Position zum Ziel, die Orientierung sowie Linear- und Winkelgeschwindigkeit kodiert (vgl. Abschnitt 4.2.1), und ein normalisiertes Tiefenbild $\mathbf{D}_t \in [0, 1]^{64 \times 64}$ einer vertikal nach unten gerichteten Kamera mit 90° Sichtfeld. Der Zustandsvektor liefert die Eigenwahrnehmung der Drohne, das Tiefenbild die einzige Information über Hindernisse. Beide Größen werden direkt und unverrauscht aus der Simulation erhoben.

Entscheidung. Zustandsvektor und Tiefenbild werden der Actor-Critic-Architektur übergeben (vgl. Abschnitt 4.3). Ein geteilter CNN-Encoder extrahiert Hindernismerkmale aus dem Tiefenbild; der resultierende Merkmalsvektor wird mit dem normalisierten Zustandsvektor konkateniert und an die MLP-Köpfe (Multilayer Perceptron) von Actor und Critic weitergegeben. Der Actor sampelt eine vierdimensionale Aktion $\mathbf{a}_t \in [-1, 1]^4$, die in einen Positionsversatz und einen Sollgierwinkel skaliert wird (vgl. Abschnitt 4.2.2).

Ausführung. Der kaskadierende PID-Regler (vgl. Abschnitt 4.6) übernimmt die skalierten Sollwerte und regelt die Drohne über 300 Simulationsschritte (3,0 s) zum Sollzustand. Anschließend erhebt die Simulation den neuen Zustand, und der Zyklus beginnt erneut. Der Agent muss somit ausschließlich lernen, *wohin* die Drohne fliegen soll; die Flugstabilisierung übernimmt der Regler.

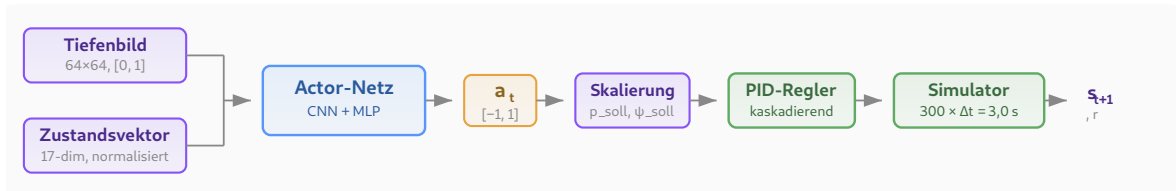


Abbildung 4.1.: Systemarchitektur mit zwei Steuerungsebenen: Der RL-Agent (oben) erhält Tiefenbild und Zustandsvektor, sampelt eine Aktion und gibt Sollwerte vor. Der PID-Regler (unten) setzt diese über 300 Simulationsschritte in Motordrehzahlen um.

4.2 Formulierung als Markov-Entscheidungsprozess

Die autonome Drohnenlandung in einem vertikalen Korridor mit Hindernissen wird als endlicher, episodischer Markov-Entscheidungsprozess (MDP) formalisiert (vgl. Abschnitt 2.1.1). Das MDP ist durch das Tupel $(\mathcal{S}, \mathcal{A}, p, r, \gamma)$ definiert, wobei $\mathcal{S}$ den Zustandsraum, $\mathcal{A}$ den Aktionsraum, p die Übergangsdynamik, r die Belohnungsfunktion und $\gamma = 0,99$ den Diskontierungsfaktor bezeichnen.

4.2.1 Zustandsraum

Der Zustand $s_t \in \mathcal{S}$ zum Entscheidungszeitpunkt t setzt sich aus einem numerischen Zustandsvektor $\mathbf{o}_t \in \mathbb{R}^{17}$ und einem Tiefenbild $\mathbf{D}_t \in [0, 1]^{64 \times 64}$ zusammen:

$$s_t = (\mathbf{o}_t, \mathbf{D}_t). \quad (4.1)$$

Der Zustandsvektor

$$\mathbf{o}_t = (\tilde{\mathbf{p}}_{\text{rel},t}, \mathbf{q}_t, \tilde{\mathbf{v}}_t^B, \tilde{\omega}_t^B, \mathbf{a}_{t-1}) \quad (4.2)$$

enthält die in Tabelle 4.1 aufgeschlüsselten Größen. Die Tilde kennzeichnet skalierte und auf $[-1, 1]$ geklippte Werte; die Skalierungsfaktoren normieren die physikalischen Bereiche auf eine einheitliche Größenordnung.

Tabelle 4.1.: Komponenten des Zustandsvektors $\mathbf{o}_t \in \mathbb{R}^{17}$.

Größe	Dim.	Beschreibung	Skalierung
$\tilde{\mathbf{p}}_{\text{rel},t}$	3	Relative Position zum Ziel	$\times \frac{1}{15}, \in [-1, 1]$
$\mathbf{q}_t$	4	Orientierungsquaternion (w, x, y, z)	—
$\tilde{\mathbf{v}}_t^B$	3	Lineargeschwindigkeit	$\times 0,4, \in [-1, 1]$
$\tilde{\omega}_t^B$	3	Winkelgeschwindigkeit	$\times \frac{1}{\pi}, \in [-1, 1]$
$\mathbf{a}_{t-1}$	4	Aktion des vorherigen Zeitschritts	—

Die relative Position $\tilde{\mathbf{p}}_{\text{rel},t} = \min(\max(\frac{1}{15}(\mathbf{p}_{\text{ziel}} - \mathbf{p}_t), -1), 1)$ kodiert Abstand und Richtung zum Landeziel, wobei der Skalierungsfaktor auf die maximale Diagonaldistanz zum Ziel ausgelegt ist. Die Lineargeschwindigkeit wird mit dem Faktor 0,4 skaliert, sodass der typische Geschwindigkeitsbereich der Drohne auf $[-1, 1]$ abgebildet wird; die Winkelgeschwindigkeit wird durch π dividiert und erreicht damit bei einer vollen Drehung pro Sekunde den Sättigungswert.

Das Tiefenbild $\mathbf{D}_t \in [0, 1]^{64 \times 64}$ wird von einer am Drohnenkörper vertikal nach unten gerichteten Kamera mit 90° Sichtfeld erzeugt:

$$\mathbf{D}_t = \min\left(\max\left(\frac{\mathbf{d}_{\text{roh}}}{d_{\text{max}}}, 0\right), 1\right), \quad d_{\text{max}} = 20 \text{ m}. \quad (4.3)$$

Das Tiefenbild liefert die einzige Information über Lage und Geometrie der Hindernisse; der numerische Zustandsvektor enthält keine explizite Hindernisrepräsentation.

4.2.2 Aktionsraum

Der Aktionsraum ist vierdimensional und kontinuierlich:

$$\mathbf{a}_t = (a_x, a_y, a_z, a_\psi) \in [-1, 1]^4. \quad (4.4)$$

Die ersten drei Komponenten definieren einen Positionsversatz relativ zur aktuellen Drohneposition:

$$\mathbf{p}_{\text{soll}} = \mathbf{p}_t + \begin{pmatrix} a_x \cdot \sigma_x \\ a_y \cdot \sigma_y \\ a_z \cdot \sigma_z \end{pmatrix}, \quad \boldsymbol{\sigma} = (1,0; 1,0; 2,0) \text{ m}. \quad (4.5)$$

Die erhöhte z -Skala ermöglicht größere vertikale Schritte, die für den vorwiegend vertikalen Abstieg erforderlich sind. Die vierte Komponente legt den Sollgierwinkel fest:

$$\psi_{\text{soll}} = a_\psi \cdot 180^\circ. \quad (4.6)$$

Die Sollwerte $(\mathbf{p}_{\text{soll}}, \psi_{\text{soll}})$ werden nicht direkt an die Motoren übergeben, sondern von einem kaskadierenden PID-Regler in Motordrehzahlen umgesetzt (vgl. Abschnitt 4.6). Diese Entkopplung reduziert den Aktionsraum von vier Motordrehzahlen auf vier semantisch interpretierbare Navigationskommandos.

4.2.3 Übergangsmodell

Die Übergangsdynamik $p(s_{t+1} \mid s_t, \mathbf{a}_t)$ wird deterministisch durch den Physiksimulator bestimmt. Der Simulator integriert die Physik mit einer Schrittweite von $\Delta t = 0,01 \text{ s}$ (100 Hz). Der RL-Agent trifft nicht bei jedem Simulationsschritt eine Entscheidung, sondern alle 300 Schritte, entsprechend einem Entscheidungsintervall von 3,0 s. Zwischen zwei Entscheidungen steuert ein kaskadierender PID-Regler die Drohne zum Sollzustand $(\mathbf{p}_{\text{soll}}, \psi_{\text{soll}})$.

4.2.4 Belohnungsfunktion

Eine zentrale Herausforderung beim Einsatz von Deep Reinforcement Learning in der Robotik liegt in der Spärlichkeit der Belohnungssignale: Der Agent muss große Zustands- und Aktionsräume explorieren, erhält jedoch nur selten informatives Feedback (Polvara u. a. 2020). Die Belohnungsfunktion kombiniert daher dichte Signale, die den Fortschritt zur Zielerreichung messen, mit spärlichen terminalen Signalen, die das eigentliche Ziel kodieren. Die Fortschrittskomponente ist dabei als potentialbasiertes Reward Shaping im Sinne von Ng

u. a. (1999) konstruiert ($\Phi(s) = -d(s)$); die übrigen dichten Komponenten weichen bewusst davon ab, da in Vorversuchen eine rein potentialbasierte Belohnungsfunktion zu langsamerer Konvergenz führte.

Für die Aufgabe des autonomen Landens mit Hindernisvermeidung wurden folgende Entwurfsprinzipien berücksichtigt:

- Die Drohne soll sicher fliegen und Hindernisse meiden.
- Die Drohne soll den kürzesten Weg zum Ziel nehmen.
- Die Drohne muss in der Nähe des Ziels zum Stillstand kommen.

Die resultierende Belohnungsfunktion setzt sich als gewichtete Summe aus fünf Komponenten zusammen:

$$r_t = w_{\text{prog}} r_{\text{prog}} + w_{\text{nah}} r_{\text{nah}} + w_{\text{prox}} r_{\text{prox}} + w_{\text{crash}} r_{\text{crash}} + w_{\text{erf}} r_{\text{erf}} \quad (4.7)$$

Dabei wird jede Komponente am Übergang (s_t, a_t, s_{t+1}) berechnet; $d_t = \|\mathbf{p}_{\text{ziel}} - \mathbf{p}_t\|$ bezeichnet den euklidischen Abstand zum Ziel vor und d_{t+1} den Abstand nach Ausführung der Aktion.

Fortschritt. Die Fortschrittsbelohnung misst die Annäherung an das Ziel zwischen zwei aufeinanderfolgenden Zeitschritten:

$$r_{\text{prog}} = d_t - d_{t+1}. \quad (4.8)$$

Ein positiver Wert zeigt an, dass sich die Drohne dem Ziel genähert hat. Bei direktem Abstieg zum Ziel beträgt $r_{\text{prog}} \approx 2,0$ pro Zeitschritt.

Nähe. Die Nähebelohnung liefert ein exponentiell ansteigendes Signal, das in der Nähe des Ziels signifikant wird:

$$r_{\text{nah}} = \exp(-d_{t+1}). \quad (4.9)$$

Bei Abständen über ca. 5 m ist r_{nah} vernachlässigbar ($< 0,01$); am Ziel nähert sich der Wert 1. Die Komponente belohnt das Verbleiben in der Nähe des Ziels.

Hindernisproximität. Die Hindernisproximität bestraft das Eindringen in einen Sicherheitsradius $d_s = 3,0$ m um Hindernisse:

$$r_{\text{prox}} = \max\left(1 - \frac{d_{\text{min}}}{d_s}, 0\right), \quad (4.10)$$

wobei $d_{\min}$ den minimalen Abstand zur achsenparallelen Begrenzungsbox (AABB) des nächsten Hindernisses bezeichnet. Die Strafe steigt linear mit abnehmendem Abstand. Der Sicherheitsradius wurde empirisch ermittelt.

Absturz. Die Absturzstrafe wird bei Verletzung einer Abbruchbedingung (vgl. Abschnitt 4.2.5) ausgelöst:

$$r_{\text{crash}} = \begin{cases} 1 & \text{Hinderniskollision, Bodenkollision, Kontrollverlust} \\ & \text{oder Verlassen des Korridors,} \\ 0 & \text{sonst.} \end{cases} \quad (4.11)$$

Erfolg. Der Erfolgsbonus wird bei Erreichen des Landeziels vergeben:

$$r_{\text{erf}} = \begin{cases} 1 & \text{Landung innerhalb des Zielradius,} \\ 0 & \text{sonst.} \end{cases} \quad (4.12)$$

Die Gewichte wurden experimentell in mehreren Trainingsdurchläufen abgestimmt. Eine zu hohe Gewichtung der Hindernisproximität und der Absturzstrafe führt zu einer stagnierenden Strategie: Da Annäherung an das Ziel und erfolgreiche Landungen zu Beginn des Trainings selten und zufällig auftreten, dominieren die negativen Belohnungen, und der Agent lernt, auf der Stelle zu schweben statt abzustiegen. Erst eine Abschwächung dieser Strafen ermöglicht exploratives Verhalten. Umgekehrt mussten Fortschritt und Nähe gegenüber der Erfolgsbelohnung ausbalanciert werden, damit der Agent nicht dauerhaftes Fliegen gegenüber dem Beenden der Episode bevorzugt. Auf eine explizite Timeout-Strafe wurde bewusst verzichtet: Ohne eine Zeitbeobachtung im Zustandsvektor kann der Critic den Episodenabbruch nicht antizipieren, was ohne Anpassung des PPO-Algorithmus zu schlechterer Performance führt (Rudin u. a. 2022).

Tabelle 4.2 fasst die gewählten Gewichte zusammen.

Tabelle 4.2.: Gewichte der Belohnungskomponenten. Negative Gewichte kennzeichnen Strafterme.

Komponente	Symbol	Gewicht
Fortschritt	w_{prog}	1,0
Nähe	w_{nah}	2,0
Hindernisproximität	w_{prox}	-0,2
Absturz	w_{crash}	-10,0
Erfolg	w_{erf}	20,0

4.2.5 Terminierung

Eine Episode endet, wenn eine der folgenden Bedingungen eintritt:

Erfolg. Die Drohne verbleibt über die gesamte Dauer eines Entscheidungsschritts (3,0 s) innerhalb eines Radius von 0,5 m um das Landeziel. Der Radius wurde bewusst eng gewählt, um eine hohe Landegenauigkeit zu erzwingen, die den beengten Platzverhältnissen einer Landung zwischen Rebzeilen entspricht.

Absturz. Mindestens eine der folgenden Bedingungen ist erfüllt:

- Flughöhe $z < 0,2$ m (Bodenkollision),
- Neigung $> 60^\circ$ in Roll oder Pitch (Kontrollverlust),
- AAB-B-Abstand zum nächsten Hindernis $< 0,3$ m (Hinderniskollision),
- Verlassen des Korridors (Bounding-Box-Prüfung auf allen Achsen).

Timeout. Die maximale Episodenlänge von $T_{\max} = 20$ Entscheidungsschritten (60 s) ist erreicht. Timeouts werden im Vorteilsschätzer als nicht-terminale Übergänge behandelt, sodass die Wertfunktion den erwarteten Ertrag über die Episode hinaus approximiert.

4.3 Netzwerkarchitektur

PPO verwendet eine Actor-Critic-Architektur (vgl. Abschnitt 2.2.5), in der zwei Netzwerke parallel trainiert werden: Der *Actor* π_θ gibt eine stochastische Strategie aus, der *Critic* V_ϕ schätzt die State-Value-Funktion zur Berechnung des Vorteilsschätzers.

Der CNN-Encoder besteht aus drei Faltungsschichten mit aufsteigender Kanalzahl (32, 64, 128), abnehmenden Kernelgrößen (8×8 , 4×4 , 3×3) und Schrittweiten (4, 2, 1). Nach jeder Faltungsschicht folgt eine Batch-Normalisierung und eine ELU-Aktivierung. Ein abschließendes Global Max Pooling reduziert jede der 128 Feature-Maps auf einen Skalar und erzeugt so einen 128-dimensionalen Merkmalsvektor unabhängig von der räumlichen Auflösung der Eingabe. Der Zustandsvektor durchläuft vor der Konkatination eine laufende empirische Normalisierung, die Mittelwert und Varianz über alle bisherigen Beobachtungen schätzt und die heterogenen physikalischen Größen auf einheitliche Skala bringt. Der resultierende Vektor aus CNN-Merkmalen und normalisiertem Zustand wird an die MLP-Köpfe von Actor und Critic weitergegeben. Der Actor-Kopf besteht aus drei vollvernetzten Schichten mit je 64 Neuronen, der Critic-Kopf aus zwei Schichten mit je 32 Neuronen; beide verwenden ELU-Aktivierung. Actor und Critic teilen sich den CNN-Encoder; die MLP-Köpfe werden getrennt trainiert. Der Actor gibt pro Aktionsdimension einen Mittelwert μ und eine lernbare

Log-Standardabweichung $\log \sigma$ aus und parametrisiert damit eine Tanh-Gaußverteilung, deren Motivation und Formulierung in Abschnitt 4.4.1.1 erläutert werden. Der Critic gibt einen skalaren State-Value $V(s_t)$ aus. Abbildung 4.2 zeigt die vollständige Architektur mit allen Schichten.

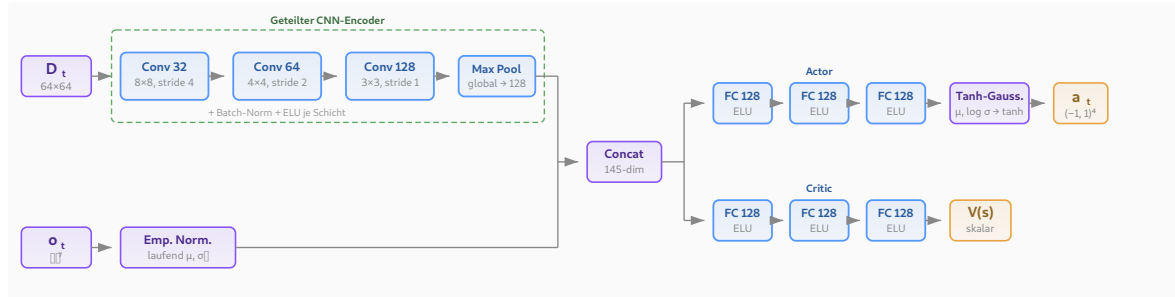


Abbildung 4.2.: Netzwerkarchitektur: Das Tiefenbild durchläuft einen geteilten CNN-Encoder (drei Faltungsschichten mit Batch-Normalisierung, ELU und abschließendem Global Max Pooling). Der resultierende 128-dimensionale Merkmalsvektor wird mit dem empirisch normalisierten Zustandsvektor konkateniert und an die getrennten MLP-Köpfe von Actor und Critic weitergegeben.

4.4 Algorithmus

4.4.1 Proximal Policy Optimization

Als Trainingsalgorithmus wird *Proximal Policy Optimization* (PPO) (Schulman, Wolski u. a. 2017) eingesetzt, dessen Clipped Surrogate Objective und vollständige Zielfunktion in Abschnitt 2.2.5 eingeführt wurden.

Die vorliegende Arbeit zielt auf einen kontinuierlichen Aktionsraum ab, in dem der Agent Zielpositionsoffsets und einen Sollgierwinkel ausgibt (vgl. Abschnitt 4.2.2). Wie in Abschnitt 2.2.5 dargelegt, übertrifft PPO auf strukturell vergleichbaren kontinuierlichen Steuerungsaufgaben die Vergleichsverfahren TRPO und A2C. Auch im spezifischen Anwendungsfeld der Drohnensteuerung hat sich PPO als Standardverfahren etabliert: Sowohl Bartolomei u. a. (2022) und Xu u. a. (2024) als auch Kaufmann, Bauersfeld, Loquercio u. a. (2023) setzen PPO ein (vgl. Abschnitt 3.3). PPO passt zudem architektonisch zum gewählten Trainings-setup: Als On-Policy-Verfahren verarbeitet es die gesammelten Daten direkt und verwirft sie nach dem Update. Off-Policy-Verfahren wie SAC (Haarnoja u. a. 2018) wurden daher verworfen. Sie erfordern einen Replay Buffer, der vergangene Transitionen speichert und bei jedem Update erneut abtastet. Ihre höhere Sample-Effizienz setzt zudem voraus, dass die Datensammlung der Engpass ist. Bei GPU-paralleler Simulation mit hunderten bis tausenden Umgebungen ist jedoch der Gradientenupdate der Engpass; die geringere Sample-Effizienz von On-Policy-Methoden wird durch den Datendurchsatz mehr als kompensiert (Rudin u. a.

2022). Neuere PPO-Varianten wie Phasic Policy Gradient (Cobbe u. a. 2021) wurden nicht evaluiert, da die verwendete Trainingsumgebung rsl-rl (Rudin u. a. 2022) ausschließlich die Standard-PPO-Implementierung bereitstellt.

4.4.1.1 Aktionsverteilung und Tanh-Squashing

Der Actor parametrisiert eine Gaußverteilung über einem unbeschränkten Hilfsraum: Für jede Aktionsdimension i wird ein Rohwert $u_i \sim \mathcal{N}(\mu_i, \sigma_i^2)$ gesampelt, wobei μ_i und $\log \sigma_i$ Netzausgaben sind. Der Aktionsraum ist jedoch auf $[-1, 1]^4$ beschränkt (vgl. Abschnitt 4.2.2). Ein naives Clipping $a_i = \text{clip}(u_i, -1, 1)$ würde die Aktionswerte korrekt begrenzen, die zugehörige Log-Wahrscheinlichkeit jedoch weiterhin auf dem unbeschnittenen Wert u_i berechnen. Da PPO das Verhältnis $r_t(\theta) = \pi_\theta(\mathbf{a}_t | s_t) / \pi_{\theta_{\text{alt}}}(\mathbf{a}_t | s_t)$ zur Gradientenschätzung nutzt (vgl. Abschnitt 2.2.5), führt diese Diskrepanz zwischen ausgeführter und bewerteter Aktion zu verzerrten Gradienten.

Statt Clipping wird daher eine invertierbare, differenzierbare Transformation verwendet (Haarnoja u. a. 2018):

$$a_i = \tanh(u_i). \quad (4.13)$$

Die zugehörige Log-Wahrscheinlichkeit ergibt sich über die Substitutionsregel für Wahrscheinlichkeitsdichten (Change of Variables):

$$\log \pi(\mathbf{a} | s) = \log \mathcal{N}(\mathbf{u} | \boldsymbol{\mu}, \boldsymbol{\sigma}^2) - \sum_{i=1}^4 \log(1 - \tanh^2(u_i)). \quad (4.14)$$

Der Korrekturterm $-\sum_i \log(1 - \tanh^2(u_i))$ ist die Log-Jacobi-Determinante der Tanh-Transformation und stellt sicher, dass die Dichte im transformierten Raum korrekt normiert bleibt.

Zusätzlich erzwingt ein Floor $\log \sigma_i \geq -2,0$ ($\sigma_i \geq 0,135$) eine Mindestexploration. Ohne diesen Floor kann PPO die Standardabweichung gegen null treiben, was in der Log-Wahrscheinlichkeit zu numerischen Instabilitäten führt und die Strategie irreversibel kollabieren lässt.

4.4.1.2 Hyperparameter

Tabelle 4.3 fasst die PPO-Hyperparameter zusammen.

Die Werte für ε , γ , λ sowie K und Gradient-Clipping entsprechen den von Schulman, Wolski u. a. (2017) empfohlenen Standardwerten, die sich in den MuJoCo-Benchmarks als robust über verschiedene Aufgaben erwiesen haben. Der GAE-Parameter $\lambda = 0,95$ folgt der

Tabelle 4.3.: PPO-Hyperparameter der vorliegenden Arbeit.

Parameter	Wert
Clipping ε	0,2
Diskontierung γ	0,99
GAE λ	0,95
Lern-Epochen K	5
Mini-Batches M	4
c_1 (Value-Loss)	1,0
c_2 (Entropie)	0,003
Gradient-Clipping	1,0

Empfehlung von Schulman, Moritz u. a. (2016) als Kompromiss zwischen Bias und Varianz. Der Entropie-Koeffizient $c_2 = 0,003$ wurde bewusst niedrig gewählt, da die Tanh-Verteilung (vgl. Abschnitt 4.4.1.1) empfindlich auf hohe Entropie-Anreize reagiert; $c_2 \geq 0,01$ führte in Pilotexperimenten zu instabilem Training.

4.5 Trainingsumgebung

4.5.1 Physiksimulator

Das Training eines RL-Agenten für die vorliegende Aufgabe erfordert einen Physiksimulator, der drei zentrale Anforderungen erfüllt: (1) GPU-parallelisierte Ausführung hunderter Umgebungsinstanzen, um die von PPO benötigten großen Rollout-Batches effizient zu erzeugen (vgl. Abschnitt 4.4); (2) simulierte Tiefenkameras, da der Agent Tiefenbilder als Beobachtung erhält; (3) Import beliebiger 3D-Objekte, um realitätsnahe Hindernisszenarien zu konstruieren.

Die vorliegende Arbeit verwendet Genesis (Genesis Authors 2024), einen 2024 veröffentlichten, quelloffenen Physiksimulator für Robotik. Genesis erfüllt alle drei definierten Anforderungen: Der Simulator führt bis zu 4096 Umgebungen GPU-parallel aus, stellt Tiefenkameras über eine Batch-Rendering-Pipeline bereit, importiert beliebige 3D-Meshes (u. a. URDF (Unified Robot Description Format), OBJ) und bietet eine durchgängige Python-API. Genesis zeichnet sich durch eine geringere Einstiegskomplexität aus: Die Installation beschränkt sich auf ein einzelnes Python-Paket, und die API erfordert keine herstellereinspezifische Toolchain.

Gegenüber verbreiteten Alternativen bietet Genesis entscheidende Vorteile: Gazebo (Koenig und Howard 2004) und PyBullet (Coumans und Bai 2016–2021) unterstützen keine GPU-parallele Ausführung hunderter Umgebungen und scheiden daher für datenintensives RL-Training aus. Isaac Gym (Makoviychuk u. a. 2021) erfüllt zwar alle genannten Anfor-

derungen, ist jedoch proprietär an NVIDIA gebunden und erfordert einen umfangreichen Installationsstack.

Einschränkend ist anzumerken, dass Genesis mit seiner Erstveröffentlichung im Dezember 2024 ein vergleichsweise junges Projekt ist und über eine kleinere Community als etablierte Alternativen verfügt.

4.5.2 Drohnenmodell

Die Drohne wird als URDF-Modell mit vier Rotoren simuliert. Die wesentlichen physikalischen Parameter sind eine Masse von 0,714 kg, ein Schub-Gewicht-Verhältnis von 2,25 sowie eine Hover-Drehzahl von 1789 RPM bei einer maximalen Drehzahl von 2700 RPM.

4.5.3 Korridorgeometrie

Die Trainingsumgebung bildet einen vertikalen Korridor der Ausdehnung $5 \times 2 \times 15$ m ($x \times y \times z$) ab (Abbildung 4.3). Der Korridor besitzt keine physischen Wände; stattdessen wird das Verlassen des Begrenzungsrahmens als Absturz gewertet (vgl. Abschnitt 4.2.5). Die Bodenebene befindet sich bei $z = 0$ m.

Die Drohne wird zu Episodenbeginn auf einer zufälligen Position im oberen Korridorbereich bei $z = 12$ m initialisiert, wobei die horizontale Position gleichverteilt innerhalb von $\pm 1,5$ m (x) bzw. $\pm 0,5$ m (y) um die Korridormitte variiert. Die Initialorientierung ist die Nulllage (Quaternion $(1, 0, 0, 0)$). Das Landeziel ist fixiert bei $\mathbf{p}_{\text{ziel}} = (0, 0, 1)^T$ m.

Während des Abstiegs durchquert die Drohne zwei horizontale Hindernisschichten bei $z \approx 2$ m und $z \approx 7$ m (vgl. Abschnitt 4.5.4).

4.5.4 Hindernisse

Als Hindernisse dienen sechs 3D-Objekte aus dem Google Scanned Objects Datensatz (GSO) (Downs u. a. 2022): eine Tasse, ein Osterkorb, ein Spielzeugdinosaurier, ein Spielzeugbus, ein Spielzeugnashorn und ein Schuh. Jedes Objekt wird um den Faktor 21 bis 63 skaliert, sodass seine horizontale Ausdehnung mindestens 2 m beträgt und damit die gesamte y -Breite des Korridors abdeckt. Objekte, deren Höhe nach der Skalierung 2,0 m überschreitet, werden von unten beschnitten.

Für die Kollisionserkennung und die Hindernisproximität (vgl. Abschnitt 4.2.4) wird die achsenparallele Begrenzungsbox (AABB) jedes Objekts herangezogen. Diese konservative Approximation umschließt das gesamte Mesh vollständig, sodass die Drohne bereits vor

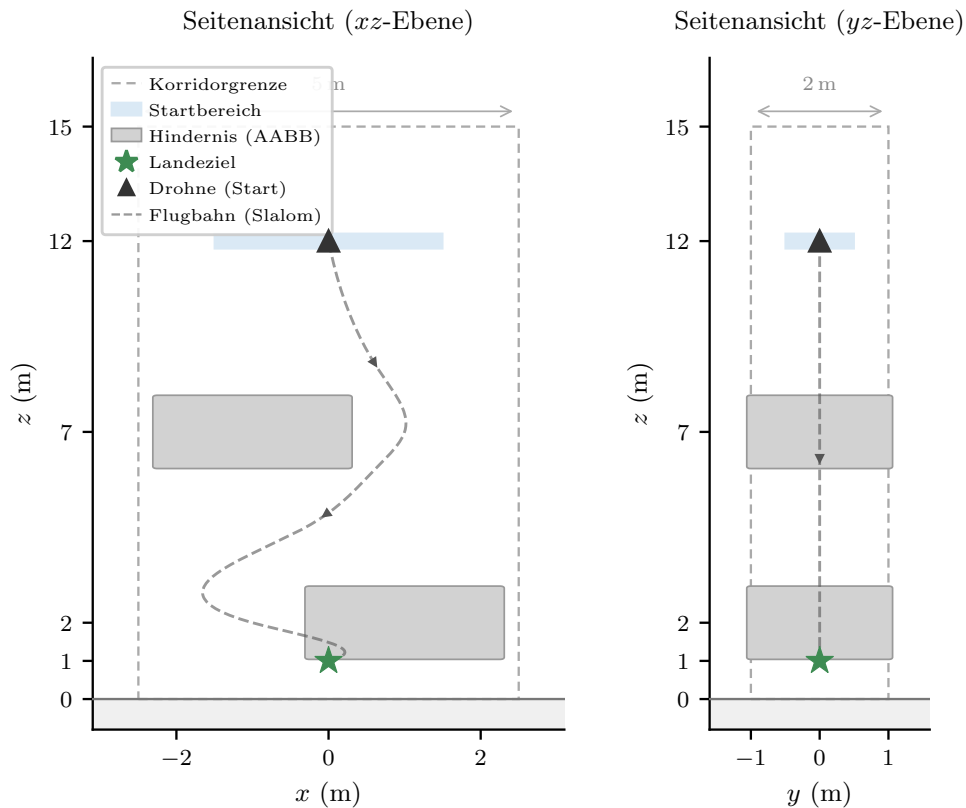


Abbildung 4.3.: Seitenansichten der Korridorgeometrie. Links: xz -Ebene (Slalom-Achse) mit schematischer Flugbahn. Rechts: yz -Ebene. Die gestrichelten Rechtecke markieren die Korridorgrenzen, graue Flächen die schematischen Hindernis-Begrenzungsboxen.

Berührung der tatsächlichen Oberfläche als kollidiert gilt. Eine oberflächenbasierte Kollisionsprüfung auf der tatsächlichen Objektgeometrie wäre exakter, erhöht jedoch den Rechenaufwand pro Simulationsschritt erheblich und war bei 512 parallelen Umgebungen im Training nicht praktikabel.

Die Platzierung folgt einem erzwungenen Slalom entlang der x -Achse: In jeder der zwei Hindernisschichten wird ein zufällig gewähltes Objekt auf einer Seite der Korridormitte positioniert, mit einem Versatz von 0,6 bis 1,2 m. Aufeinanderfolgende Schichten wechseln die Seite, sodass die Drohne einen Slalomkurs fliegen muss. Die Startseite ($+x$ oder $-x$) wird pro Episode zufällig gewählt.

4.6 Kaskadierender PID-Regler

Wie in Abschnitt 3.3 dargelegt, folgt die vorliegende Arbeit einer hierarchischen Steuerungsarchitektur: Der RL-Agent gibt Sollposition und Sollgierwinkel vor, ein nachgeschalteter Regler setzt diese in Motordrehzahlen um. Der Regler ist als dreistufige PID-Kaskade aufgebaut:

1. **Positionsregler** (äußere Schleife): Vergleicht die Sollposition mit der aktuellen Position und gibt Sollgeschwindigkeiten aus, begrenzt auf 5 m/s horizontal und 3 m/s vertikal.
2. **Geschwindigkeitsregler** (mittlere Schleife): Wandelt Geschwindigkeitsfehler in ein Schubkommando sowie gewünschte Roll- und Nickwinkel um, begrenzt auf $\pm 30^\circ$.
3. **Lageregler** (innere Schleife): Regelt Roll, Nick und Gier auf die gewünschten Winkel und gibt Korrektursignale an den Motormischer.

Der Motormischer addiert die vier Korrektursignale (Schub, Roll, Nick, Gier) zur Schwebedrehzahl 1789 RPM und verteilt sie auf die vier Motoren. Der Agent muss somit ausschließlich die Navigationsstrategie erlernen, nicht die Flugdynamik. Der Regler wurde eigens für diese Simulationsumgebung entwickelt und seine Parameter manuell abgestimmt (Tabelle D1). Ein etablierter Flugreglerstack wie PX4 stand im verwendeten Simulator nicht zur Verfügung.

4.7 Trainingsprozess

Tabelle 4.4 fasst die Trainingsparameter zusammen.

Tabelle 4.4.: Trainingsparameter.

Parameter	Wert
Parallele Umgebungen N	512
Schritte pro Umgebung T	60
Batch-Größe $N \times T$	30 720
Lernrate α	1×10^{-3}
Optimizer	Adam
Trainingsiterationen	1 000
GPU	NVIDIA A30 (24 GB)
Laufzeit	~24 h
Checkpoint-Intervall	100 Iterationen

Das Training wird auf dem HPC-Cluster (High-Performance Computing) der Universität Leipzig durchgeführt. Als GPU dient eine NVIDIA A30 (Ampere-Architektur, 24 GB VRAM). Die A30 wurde gegenüber der alternativ verfügbaren NVIDIA V100 (Volta) gewählt, da der Batch-Renderer für die Tiefenkamera eine GPU mit Compute Capability $\geq 7,5$ (Turing oder neuer) voraussetzt. Auf der V100 steht lediglich der langsamere Rasterizer-Pfad zur Verfügung, der die Tiefenbilder seriell pro Umgebung rendert.

Die Anzahl paralleler Umgebungen $N = 512$ ist durch den GPU-Speicher begrenzt. Die Rollout-Länge $T = 60$ Entscheidungsschritte bei 3,0 s Entscheidungsintervall (vgl. Abschnitt 4.2.3) ergibt eine Rollout-Dauer von 180 s, die drei vollständige Episoden ($T_{\max} = 60$ s) umfassen kann. Pro Iteration sammelt der Agent damit $N \times T = 30\,720$ Transitionen, die in

$M = 4$ Mini-Batches à 7 680 Transitionen aufgeteilt und $K = 5$ Epochen lang optimiert werden.

Die Lernrate $\alpha = 1 \times 10^{-3}$ wurde experimentell bestimmt: Kleinere Werte führten zu langsamer Konvergenz, größere zu kollabierenden Strategien. Die Lernrate wird über den gesamten Trainingsverlauf konstant gehalten. Alternativen (Cosine Decay, Linear Decay sowie die adaptive KL-Penalty-Variante nach Schulman, Wolski u. a. (2017)) wurden in Vorexperimenten getestet, zeigten jedoch keine Verbesserung gegenüber der konstanten Rate.

Zu Beginn jeder Trainingsausführung werden die Episodenlängen der N Umgebungen zufällig initialisiert, sodass die Episodenenden über die Umgebungen hinweg zeitlich versetzt auftreten. Ohne diese Dekorrelation würden alle Umgebungen synchron zurückgesetzt, was innerhalb eines Rollouts zu korrelierten Transitionsblöcken führt und die Varianz der Gradientenschätzung erhöht (Rudin u. a. 2022).

Der Agent wird über 1 000 Iterationen trainiert, was $1\,000 \times 30\,720 = 30\,720\,000$ gesammelten Transitionen entspricht. Alle 100 Iterationen wird ein Checkpoint gespeichert. Die Laufzeit beträgt circa 24 Stunden auf der genannten Hardware.

4.8 Evaluation

Die Evaluation untersucht die Leistungsfähigkeit der trainierten Strategie anhand zweier Versuchsreihen mit jeweils 1 000 Episoden und vergleicht sie mit einem RRT-Connect-Pfadplaner. Die erste Versuchsreihe verwendet die Trainingsumgebung mit zuvor unbekannten Hindernisformen; die zweite eine photogrammetrisch erfasste Agrarumgebung.

4.8.1 Evaluationsmetriken

Jede Episode wird anhand der folgenden Metriken bewertet. Alle Metriken werden mit 95 %-Bootstrap-Konfidenzintervallen versehen (Efron und Tibshirani 1994): Aus den $n = 1\,000$ Episodenergebnissen werden $B = 10\,000$ Stichproben mit Zurücklegen gezogen; für jede Stichprobe wird die jeweilige Metrik berechnet. Die Grenzen des Konfidenzintervalls ergeben sich als das 2,5 %- und 97,5 %-Perzentil der Bootstrap-Verteilung (Perzentilmethode).

Erfolgsrate. Anteil der Episoden, in denen die Drohne das Landeziel innerhalb des Zielradius (0,5 m) erreicht, ohne eine Abbruchbedingung (vgl. Abschnitt 4.2.5) auszulösen und innerhalb des Zeitlimits von 60 s.

Fehlerartenverteilung. Jede fehlgeschlagene Episode wird einer von vier Kategorien zugeordnet: Hinderniskollision, Bodenkollision, Verlassen des Korridors (nur Versuchsreihe 1),

Terrain-Kollision (nur Versuchsreihe 2, vgl. Abschnitt 4.8.3) oder Timeout. Die Verteilung gibt Aufschluss darüber, *warum* die Strategie versagt, nicht nur *ob*.

Landezeit. Zeit vom Episodenbeginn bis zur erfolgreichen Landung, berechnet ausschließlich über erfolgreiche Episoden. Die Landezeit wird als Median mit Interquartilsabstand (IQR) angegeben, da die Verteilung typischerweise rechtsschief ist: Die Mehrzahl der Episoden endet schnell, während vereinzelte Episoden mit ungünstigen Hinderniskonfigurationen deutlich länger dauern. Der Median ist gegenüber solchen Ausreißern robuster als der Mittelwert.

Minimaler Hindernisabstand. Der kleinste AABB-Abstand zwischen Drohne und einem Hindernis, gemessen über den gesamten Episodenverlauf. Angegeben werden der Mittelwert über alle Episoden sowie das globale Minimum. Diese Metrik quantifiziert die Sicherheitsmargen der Strategie und ist auch für erfolgreiche Episoden aussagekräftig: Ein niedriger mittlerer Mindestabstand deutet auf eine riskante Flugweise hin, selbst wenn keine Kollision auftritt.

Pfادهffizienz. Das Verhältnis der tatsächlichen Trajektorienlänge zur euklidischen Distanz zwischen Startposition und Landeziel:

$$\eta_{\text{Pfad}} = \frac{\sum_{t=0}^{T-1} \|\mathbf{p}_{t+1} - \mathbf{p}_t\|}{\|\mathbf{p}_{\text{ziel}} - \mathbf{p}_0\|}. \quad (4.15)$$

Ein Wert von $\eta_{\text{Pfad}} = 1,0$ entspricht einem direkten Pfad; höhere Werte zeigen Umwege an. Die Pfادهffizienz wird nur über erfolgreiche Episoden berechnet und als Median mit IQR angegeben.

Rechenzeit pro Episode. Die Summe aller Inferenzzeiten innerhalb einer erfolgreichen Episode, gemessen auf identischer Hardware für die RL-Strategie und den Pfadplaner. Für die RL-Strategie entspricht jede Inferenz einem einzelnen Forward-Pass durch das CNN-MLP-Netzwerk; die Messung erfolgt nach GPU-Synchronisation, um asynchrone Kernelausführung nicht zu unterschätzen. Die Metrik verbindet den Rechenaufwand pro Entscheidungsschritt mit der Anzahl benötigter Schritte: Ein Verfahren, das pro Schritt schneller rechnet und zugleich weniger Schritte bis zur Landung benötigt, erzielt eine niedrigere Gesamtrechenzeit. Die Angabe erfolgt als Median mit Interquartilsabstand über alle erfolgreichen Episoden.

4.8.2 Versuchsreihe 1: Generalisierung auf unbekannte Hindernisformen

Die erste Versuchsreihe prüft, ob die trainierte Strategie auf Hindernisformen generalisiert, die während des Trainings nicht gesehen wurden. Die Korridorgeometrie, die Spawnver-

teilung, das Landeziel und das Zeitlimit bleiben identisch zur Trainingsumgebung (vgl. Abschnitt 4.5.3). Einzig die sechs Hindernisobjekte werden durch sechs zuvor unbekannte 3D-Objekte aus demselben Google Scanned Objects Datensatz ersetzt. Die Ersatzobjekte werden so skaliert, dass ihre horizontale Ausdehnung derjenigen der Trainingsobjekte entspricht. Die Platzierungslogik (erzwungener Slalom, alternierende Seiten) bleibt unverändert. Die Drohne muss ihre Ausweichstrategie somit rein auf Basis der Tiefenbilder auf unbekannte Geometrien übertragen.

Als Vergleichsverfahren dient ein RRT-Connect-Pfadplaner, der auf der vollständigen Kollisionsgeometrie der Umgebung operiert: Ihm stehen die exakten Positionen und Ausdehnungen aller Hindernisse zur Verfügung. Der RL-Agent hingegen erhält ausschließlich ein lokales Tiefenbild und den propriozeptiven Zustandsvektor. Die Leistungsdifferenz zwischen beiden Verfahren quantifiziert daher den Preis eingeschränkter Sensorik, nicht den Preis der lernbasierten Methode. Beide Verfahren werden unter identischen Bedingungen evaluiert und anhand aller in Abschnitt 4.8.1 definierten Metriken verglichen.

4.8.3 Versuchsreihe 2: Transfer in eine realistische Agrarumgebung

Die zweite Versuchsreihe untersucht den Zero-Shot-Transfer der trainierten Strategie in eine photogrammetrisch erfasste Weinbergumgebung. Als Grundlage dient ein photogrammetrisches 3D-Modell eines realen Weinbergs (Eltville, Deutschland), das von der betreuenden Forschungsgruppe bereitgestellt wurde. Das Modell wird mit vierfacher Skalierung geladen, um eine dem Korridor vergleichbare räumliche Ausdehnung zu erreichen. Die Reihen und das natürliche Terrain bilden die einzigen Hindernisse; ein Korridorbegrenzungsrahmen entfällt.

Die Kollisionserkennung wird gegenüber der Trainingsumgebung angepasst: Aus dem Weinberg-Mesh wird vorab ein zweidimensionales Höhenraster berechnet, das die Geländehöhe an jedem Gitterpunkt speichert. Zur Laufzeit wird die Drohnenhöhe mit der interpolierten Geländehöhe unter ihr verglichen; unterschreitet der vertikale Abstand einen Sicherheitsabstand von 0,5 m, wird die Episode als Terrain-Kollision gewertet. Die übrigen Abbruchbedingungen (Kontrollverlust, Timeout) bleiben identisch; die AABB-basierte Hinderniskollision und die Korridorgrenzenprüfung aus Abschnitt 4.2.5 entfallen.

Als Landeziele dienen 20 vordefinierte Positionen: 15 in den Bodenstreifen zwischen Reihen und 5 auf offenem Gelände. Pro Episode wird eine Position zufällig gewählt. Die Mehrzahl der Ziele liegt damit in unmittelbarer Nähe von Vegetation; im Gegensatz zur

Trainingsumgebung, in der die Landezone stets hindernisfrei ist, muss die Drohne hier in beengter Umgebung landen. Abbildung 4.4 zeigt die räumliche Verteilung der Zielpositionen auf dem Höhenraster. Die Drohne wird in variabler Höhe (10 bis 15 m) über dem Landeziel initialisiert.

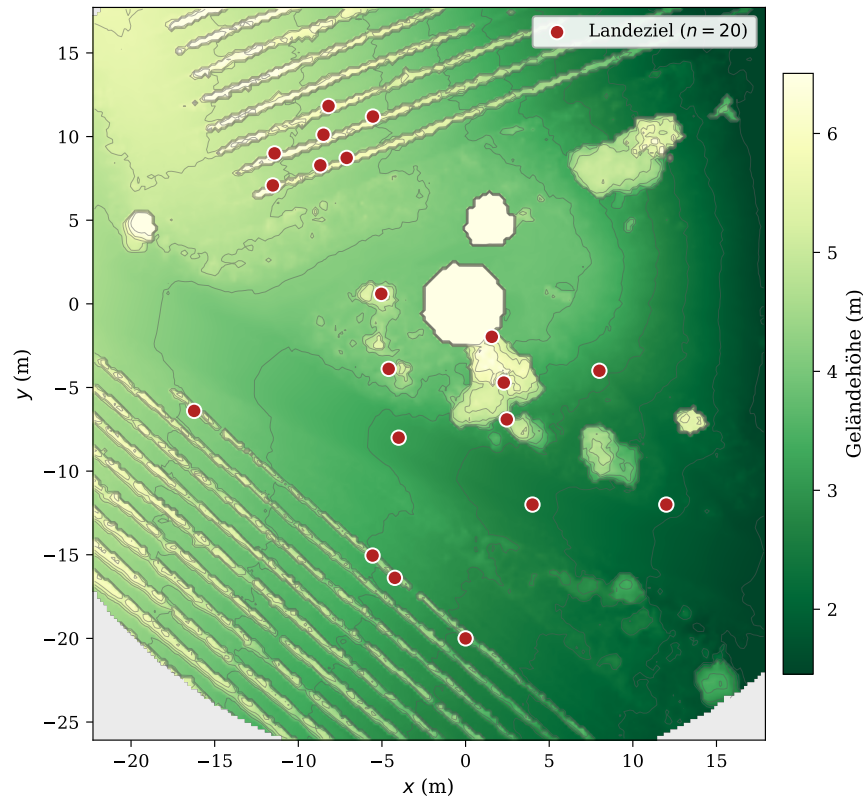


Abbildung 4.4.: Draufsicht auf das Höhenraster der Weinbergumgebung (4-fache Skalierung) mit den 20 vordefinierten Zielpositionen. Die diagonalen Strukturen entsprechen den Rebenreihen; die Mehrzahl der Ziele liegt in den Bodenstreifen dazwischen.

Die Evaluation erfolgt mit denselben Metriken und demselben Konfidenzintervallverfahren aus Abschnitt 4.8.1. Als Vergleichsverfahren dient ein RRT-Connect-Pfadplaner, der auf demselben Höhenraster operiert und damit über vollständige Geländekenntnis verfügt.

5 Ergebnisse

Dieses Kapitel präsentiert die experimentellen Ergebnisse des Korridornavigationsagenten. Abschnitt 5.1 beschreibt den Trainingsverlauf, Abschnitt 5.2 und 5.3 die Evaluationsergebnisse in der Korridorumgebung und im Weinberg-Transfer, und Abschnitt 5.4 den Vergleich mit dem klassischen RRT-Planer.

5.1 Trainingsverhalten

Das Training wurde mit 512 parallelen Umgebungen auf einer NVIDIA A30 GPU durchgeführt und erreichte einen durchschnittlichen Durchsatz von 341 FPS. Der Trainingslauf umfasst 957 Iterationen und wurde nach 24 Stunden durch die GPU-Zeitbegrenzung des HPC-Clusters beendet. Abbildung 5.1 zeigt den Verlauf der kumulativen Belohnung.

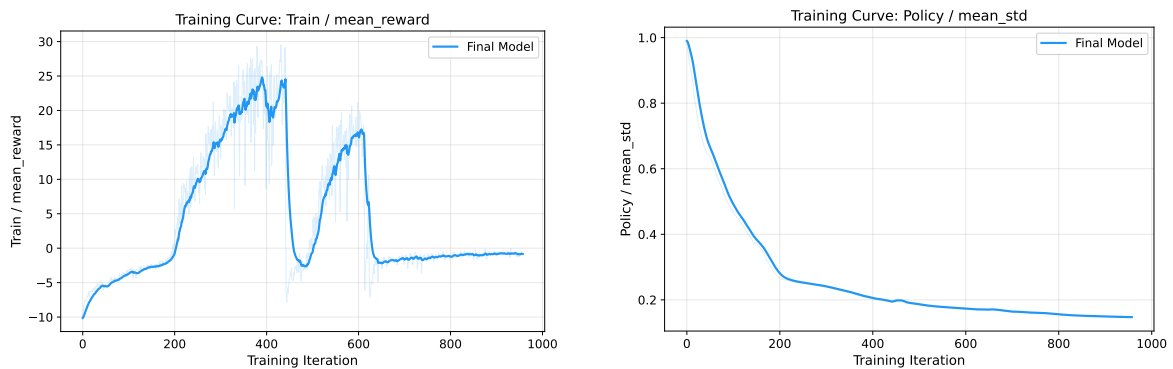


Abbildung 5.1.: Kumulative Belohnung (links) und mittlere Standardabweichung der Aktionsverteilung (rechts) über 957 Trainingsiterationen. Der monotone Abfall der Standardabweichung zeigt die Konvergenz zu einer deterministischen Strategie. Geglättet mit EMA $\alpha = 0,9$, transparenter Hintergrund: Rohwerte.

Der Trainingsverlauf lässt sich in zwei Phasen unterteilen. In der ersten Phase (Iteration 0 bis 441) steigt die kumulative Belohnung kontinuierlich an. Der Agent löst dabei nacheinander zwei Teilaufgaben: Zunächst lernt er die Zielnavigation und den vertikalen Abstieg durch die Hindernisschichten, wobei die Absturzrate zunächst hoch bleibt (Abbildung 5.2, links). Ab circa Iteration 200 erscheint erstmals die Erfolgsbelohnung (Abbildung 5.2, rechts), und gleichzeitig sinkt die Absturzrate. Die kumulative Belohnung überschreitet die Null-Grenze bei Iteration 203 und erreicht ihren Höchstwert von 24,8 bei Iteration 390. Die Standardabweichung der Aktionsverteilung (Abbildung 5.1, rechts) fällt monoton von 0,99 auf 0,15, was zeigt, dass die Strategie zu einer deterministischen Strategie konvergiert und nicht zufällig agiert. Die übrigen Belohnungskomponenten bestätigen den Lernfortschritt (Abbildungen A1 und A2 im Anhang).

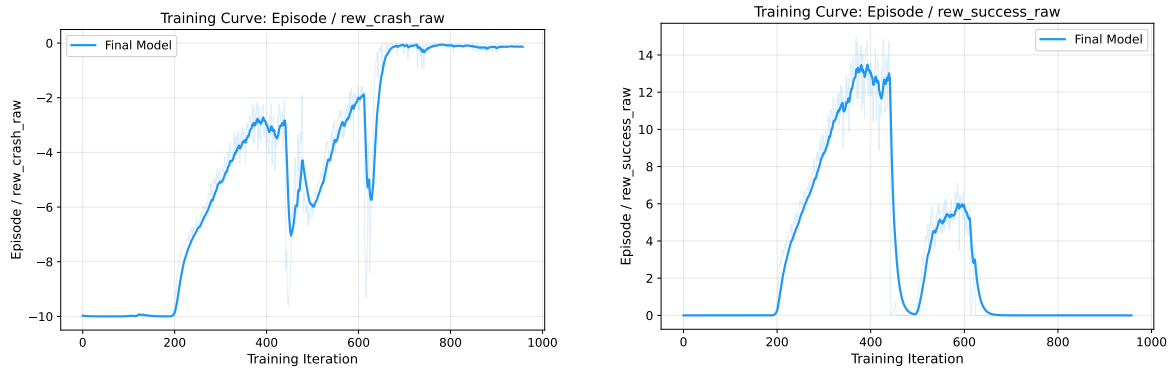


Abbildung 5.2.: Unskalierte Absturzstrafe (links) und Erfolgsbelohnung (rechts). Die Absturzstrafe spiegelt die Häufigkeit von Abstürzen pro Episode, die Erfolgsbelohnung die erreichte Verweilzeit im Zielbereich. Geglättet mit EMA $\alpha = 0,9$.

Ab Iteration 442 bricht die kumulative Belohnung abrupt ein: Innerhalb einer einzigen Iteration fällt sie von 29,2 auf $-6,1$. Die Standardabweichung der Aktionsverteilung bleibt dabei nahezu unverändert ($0,193 \rightarrow 0,194$); der Kollaps betrifft die Mittelwerte der Strategie, nicht die Explorationsbreite. Ein Erholungsversuch um Iteration 600 (Belohnung ~ 17) bleibt instabil; bis zum Trainingsende pendelt die Belohnung um -1 .

Tabelle 5.1 fasst die Laufzeiten zusammen. Der beste Checkpoint liegt bei Iteration 400 (10,4 Stunden Trainingszeit, kumulative Belohnung 23,0). Auf Basis des Belohnungsverlaufs wurde dieser Checkpoint für alle nachfolgenden Evaluierungen ausgewählt (*Early Stopping*). Die verbleibenden 14 Stunden Trainingszeit (Iteration 400 bis 957) produzieren kein Modell, das die Leistung von Checkpoint 400 übertrifft.

Tabelle 5.1.: Konvergenzgeschwindigkeit und Checkpoint-Selektion (512 Umgebungen, NVIDIA A30, 341 FPS).

Checkpoint	Iteration	Laufzeit	Kum. Belohnung
Bestes Modell	400	10,4 h	23,0
Post-Kollaps	900	22,6 h	$-1,1$
Trainingsende	957	24,0 h	$-0,8$

5.2 Phase 1: Korridornavigation

Versuchsreihe 1 (vgl. Abschnitt 4.8.2) evaluiert die trainierte Strategie (Checkpoint 400) über 1 000 Episoden in der Korridorumgebung mit zuvor ungesesehenen Hindernisobjekten.

Der Agent erreicht eine Erfolgsrate von 71,0 %. Die fehlgeschlagenen Episoden verteilen sich nahezu vollständig auf eine einzige Fehlerart: Hinderniskollisionen verursachen 28,8 % aller Episoden (288 von 1 000). Die verbleibenden 0,2 % (2 Episoden) enden durch Ver-

lassen des Korridorbegrenzungsrahmens. Bodenkollisionen und Timeouts treten nicht auf. Abbildung 5.3 zeigt die Ergebnisse.

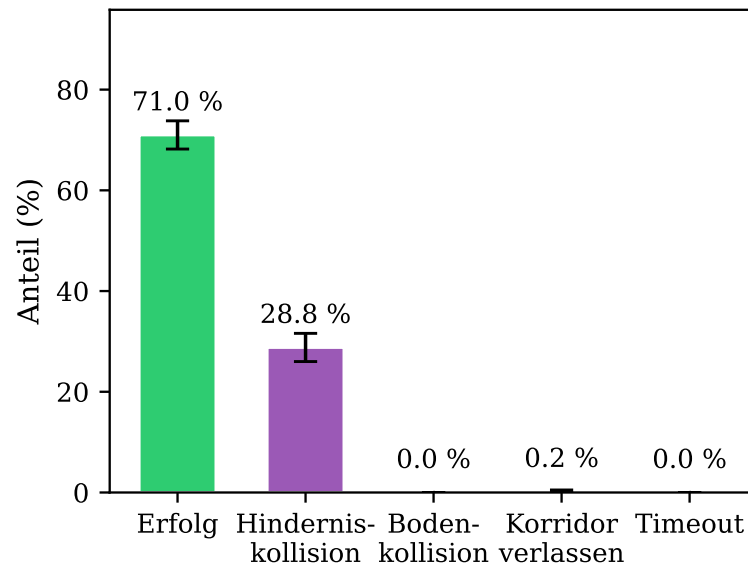


Abbildung 5.3.: Episodenergebnisse der Phase-1-Evaluation (1 000 Episoden). Fehlerbalken zeigen 95 %-Bootstrap-Konfidenzintervalle.

Tabelle B1 im Anhang listet alle Metriken mit Konfidenzintervallen.

Abbildung 5.4 zeigt drei exemplarische Trajektorien erfolgreicher Episoden. Das Flugverhalten lässt sich in zwei wiederkehrende Phasen unterteilen:

(1) Vertikaler Abstieg bei freier Bahn. In Abschnitten ohne Hindernisse, sowohl oberhalb der ersten als auch unterhalb der letzten Hindernisschicht, vollzieht der Agent einen annähernd vertikalen Abstieg in Richtung des Landeziels. Die mediane Pfadeffizienz von 1,19 (Tabelle B1) bestätigt, dass die Trajektorien insgesamt nahe am direkten Pfad verlaufen.

(2) Abbremsen und laterales Ausweichen. In der Nähe eines Hindernisses verlangsamt der Agent den Abstieg und geht in ein Schweben über. Das Abbremsen erfolgt spät; die anschließenden horizontalen Ausweichmanöver verlaufen langsam und mit geringem Sicherheitsabstand (mittlerer Minimalabstand 0,52 m, Tabelle B1). Über den gesamten Episodenverlauf treten zusätzlich Gierrotationen auf, die keinem erkennbaren navigatorischen Zweck dienen.

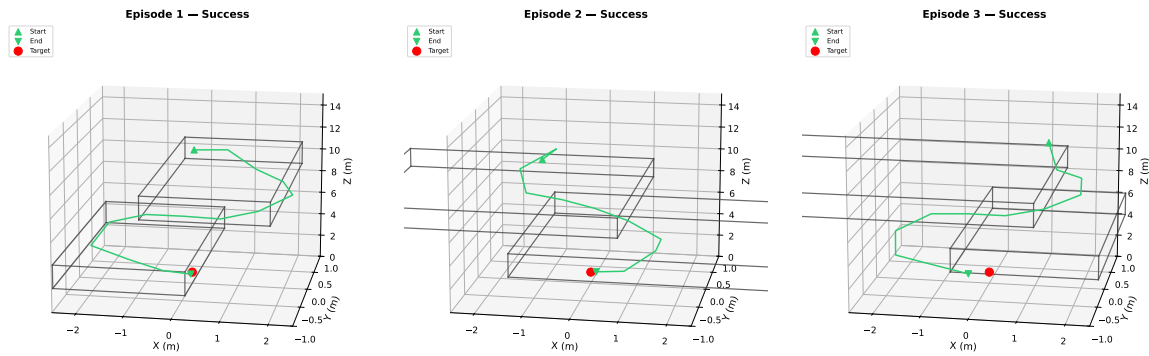


Abbildung 5.4.: Exemplarische 3D-Trajektorien dreier erfolgreicher Episoden in der Korridorumgebung. Die grauen Quader zeigen die achsenparallelen Begrenzungsboxen (AABB) der Hindernisse. Grünes Dreieck: Startposition bei $z \approx 12$ m, roter Punkt: Landeziel bei $(0, 0, 1)$ m.

5.2.1 Nachuntersuchung: Oberflächenbasierte Kollisionserkennung

Das knappe Passieren der Hindernisse bei gleichzeitig hoher Kollisionsrate legte nahe, dass die konservative AABB-Kollisionserkennung (vgl. Abschnitt 4.5.4) einen Teil der Fehlschläge verursacht. Um dies zu prüfen, wurde eine Nachuntersuchung mit 100 Episoden durchgeführt, in der die Kollisionserkennung durch die geometrisch exakte Oberflächenkollisionsprüfung des Simulators ersetzt wurde.

Mit oberflächenbasierter Kontakterkennung steigt die Erfolgsrate auf 98,0 % (Tabelle 5.2). Ein erheblicher Anteil der unter der AABB-Metrik registrierten Kollisionen sind demnach Fehlalarme: Die Drohne durchfliegt die Begrenzungsbox, ohne die tatsächliche Objektoberfläche zu berühren.

Tabelle 5.2.: Vergleich der Evaluationsergebnisse mit AABB-Distanz ($n = 1\,000$) und oberflächenbasierter Kontakterkennung ($n = 100$).

Metrik	AABB ($n = 1\,000$)	Oberfläche ($n = 100$)
Erfolgsrate	71,0 %	98,0 %
Hinderniskollision	28,8 %	2,0 %

5.3 Phase 2: Weinberg-Transfer

Versuchsreihe 2 (vgl. Abschnitt 4.8.3) evaluiert den Zero-Shot-Transfer der trainierten Strategie (Checkpoint 400) in die Weinbergumgebung über 1 000 Episoden.

Der Agent erreicht eine Erfolgsrate von 10,4 %. Die fehlgeschlagenen Episoden verteilen sich auf zwei Fehlerarten: Terrain-Kollisionen verursachen 15,9 % aller Episoden, Timeouts 73,7 %. Abbildung 5.5 zeigt die Ergebnisse.

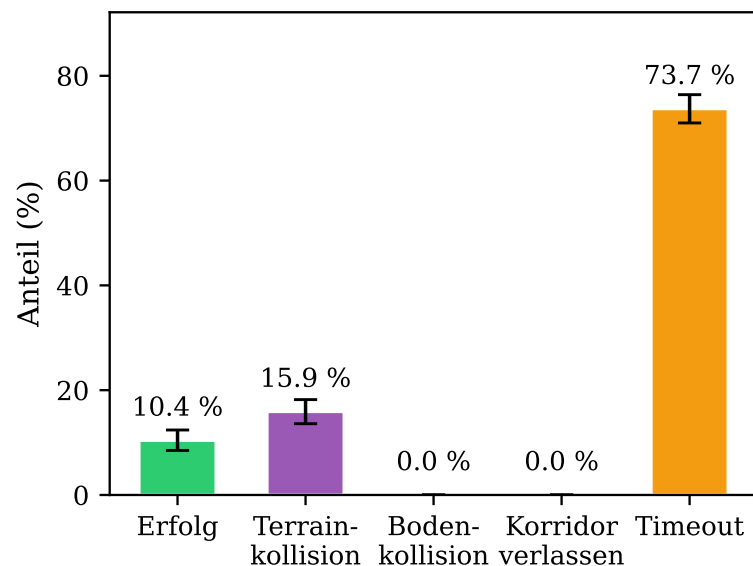


Abbildung 5.5.: Episodenergebnisse der Phase-2-Evaluation (1 000 Episoden, Zero-Shot-Transfer in die Weinbergumgebung). Fehlerbalken zeigen 95 %-Bootstrap-Konfidenzintervalle.

Die dominante Fehlerart ist das Timeout: 73,7 % aller Episoden (737 von 1 000) enden durch Überschreitung des Zeitlimits von 60 s, ohne dass eine Kollision oder eine erfolgreiche Landung eintritt. Der mediane Abstand zum Landeziel am Ende der Episode beträgt 1,19 m (Tabelle C1). Von den 737 Timeout-Episoden befinden sich 440 (59,7 %) zum Zeitpunkt des Abbruchs innerhalb von 1,5 m zum Landeziel.

Tabelle C1 im Anhang listet alle Metriken mit Konfidenzintervallen.

Das beobachtete Flugverhalten lässt sich in zwei wiederkehrende Muster unterteilen:

- (1) **Direkter Abstieg bei freier Bahn.** Bei Szenarien ohne Vegetation zwischen Drohne und Landeziel vollzieht der Agent einen annähernd direkten Abstieg, vergleichbar mit Phase 1.
- (2) **Schweben nahe dem Ziel ohne Konvergenz.** In der Mehrzahl der Fälle erreicht der Agent die Umgebung des Landeziels, verbleibt jedoch außerhalb des Erfolgsradius von 0,5 m, teils schwebend über nahegelegener Vegetation, teils mit ineffektiven Korrekturbewegungen. Die Feinsteuerungsmanöver für die letzten Meter fehlen.

5.3.1 Sensitivitätsanalyse des Erfolgskriteriums

Der hohe Timeout-Anteil bei gleichzeitig geringem Abstand zum Landeziel legte nahe, dass der strikte Erfolgsradius von 0,5 m einen erheblichen Anteil der Fehlschläge verursacht. Um die Sensitivität der Erfolgsrate gegenüber dem Erfolgskriterium zu quantifizieren, wurde eine Nachuntersuchung mit 1 000 Episoden und einem erweiterten Erfolgsradius von 1,5 m durchgeführt.

Mit dem erweiterten Erfolgsradius steigt die Erfolgsrate auf 69,7 % (Tabelle 5.3). Die Terrain-Kollisionsrate sinkt von 15,9 % auf 9,4 %, da ein Teil der zuvor als Timeout gewerteten Episoden nun frühzeitig als Erfolg beendet wird: Die Drohne verbringt weniger Zeit in der Nähe von Vegetation und ist dadurch einem geringeren Kollisionsrisiko ausgesetzt. Der Timeout-Anteil sinkt von 73,7 % auf 27,7 %.

Tabelle 5.3.: Sensitivitätsanalyse des Erfolgsradius r in der Weinbergumgebung
($n = 1\,000$ Episoden je Bedingung). Alle übrigen Parameter bleiben identisch.

Metrik	$r = 0,5\text{ m}$	$r = 1,5\text{ m}$
Erfolgsrate	10,4 %	69,7 %
Terrain-Kollision	15,9 %	9,4 %
Timeout	73,7 %	27,7 %

5.4 Vergleich RL vs. RRT-Connect

Dieser Abschnitt stellt die Ergebnisse des RL-Agenten und des RRT-Connect-Pfadplaners (vgl. Abschnitt 4.8.2) über beide Versuchsreihen gegenüber. Tabelle 5.4 fasst alle Metriken zusammen.

Tabelle 5.4.: Vergleich von RL-Agent und RRT-Connect über beide Versuchsreihen
($n = 1\,000$ Episoden je Bedingung).

Metrik	Phase 1 (Korridor)		Phase 2 (Weinberg)	
	RL	RRT	RL	RRT
Erfolgsrate	71,0 %	82,0 %	10,4 %	85,7 %
(relaxed $r = 1,5\text{ m}$)	n. a.		69,7 %	98,4 %
Hindernis-/Terrain-Koll.	28,8 %	18,0 %	15,9 %	7,1 %
Timeout	0,0 %	0,0 %	73,7 %	7,9 %
Landezeit (Median)	30,0 s	42,0 s	36,0 s	42,0 s
Pfadeffizienz (Median)	1,19	1,09	1,07	0,99
Min. Abstand (Mittel)	0,52 m	0,48 m	1,02 m	0,46 m
Rechenzeit/Ep. (Median)	0,21 ms	80,4 ms	0,25 ms	18,7 ms

Der RRT-Planer übertrifft den RL-Agenten in der Erfolgsrate beider Umgebungen, wobei der Abstand im Weinberg deutlich größer ausfällt (85,7 % gegenüber 10,4 %). Im Korridor dominieren bei beiden Verfahren Hinderniskollisionen als Fehlerursache; im Weinberg hingegen scheitert der RL-Agent überwiegend an Timeouts (73,7 %), während der RRT-Planer diese auf 7,9 % begrenzt.

Der RL-Agent landet in beiden Umgebungen schneller als der RRT-Planer (30,0 s gegenüber 42,0 s im Korridor). Die Pfadeffizienz ist hingegen beim RRT-Planer höher (1,09 gegenüber

1,19 im Korridor), da dieser den kürzesten kollisionsfreien Pfad vor Flugbeginn berechnet, während der RL-Agent reaktiv ausweicht und dadurch längere Trajektorien fliegt.¹ Der deutlichste Unterschied zeigt sich in der Rechenzeit zur Einsatzzeit: Ein Forward-Pass des neuronalen Netzes benötigt 0,21 ms, während RRT-Connect im Korridor 80,4 ms pro Episode für die Pfadplanung benötigt. Dem steht jedoch ein einmaliger Trainingsaufwand von circa 24 Stunden auf einer NVIDIA A30 gegenüber, den der RRT-Planer nicht erfordert. Detaillierte Metriken mit Konfidenzintervallen finden sich in den Anhangstabellen B1 und C1.

¹Einzelne Werte knapp unter 1,0 sind ein Implementierungsfehler in der Messung: Das letzte Flugsegment wird aufgrund des automatischen Zurücksetzens der Umgebung nicht erfasst, und eine erfolgreiche Episode endet am Rand des Erfolgsradius, nicht am Zielpunkt selbst.

6 Diskussion

Dieses Kapitel interpretiert die in Kapitel 5 berichteten Ergebnisse im Kontext der vier Nebenfragestellungen. Jeder Abschnitt schließt mit einer expliziten Antwort auf die zugehörige Fragestellung.

6.1 Leistungsfähigkeit in der Trainingsumgebung

6.1.1 Trainingsverhalten und Konvergenz

Die zwei beobachteten Phasen im Belohnungsverlauf lassen sich auf die Belohnungsstruktur zurückführen, die dichte Komponenten mit ereignisbasierten Boni kombiniert. In der Konvergenzphase (Iteration 0 bis 441) ermöglichen die dichten Komponenten (r_{prog} , r_{nah} , r_{prox}) den initialen Anstieg, indem sie bereits für Teilfortschritte Signal liefern. Ab circa Iteration 200 setzt zusätzlich die Erfolgsbelohnung r_{erf} ein, sobald der Agent erstmals konsistente Landungen erreicht, und beschleunigt den Belohnungsanstieg. Ohne die dichten Komponenten würde dem Agenten in der frühen Trainingsphase das Gradientensignal für Teilfortschritte fehlen, was die Konvergenz vermutlich erheblich verzögern würde.

Der abrupte Kollaps ab Iteration 442 betrifft ausschließlich die Mittelwerte der Strategie bei nahezu unveränderter Standardabweichung ($0,193 \rightarrow 0,194$), was auf eine katastrophale Strategieaktualisierung hindeutet, nicht auf einen graduellen Stabilitätsverlust. Obwohl der Clipping-Mechanismus von PPO (Schulman, Wolski u. a. 2017) große Strategieänderungen pro Iteration begrenzen soll, reicht er nicht aus, um solche abrupten Einbrüche zu verhindern. Eine mögliche Erklärung bieten Moalla u. a. (o. D.), die zeigen, dass eine schleichende Verschlechterung des Feature-Ranks im Actor-Netzwerk den Clipping-Mechanismus unterlaufen und zu unumkehrbaren Leistungseinbrüchen führen kann, wobei die Standardabweichung als globaler Parameter unbeeinflusst bleibt. Ob dieser Mechanismus den beobachteten Kollaps erklärt, lässt sich ohne Analyse des Feature-Ranks nicht abschließend beurteilen. Als Eine schrittweise Reduktion der Lernrate (linear und kosinusförmig) sowie ein Weitertraining mit reduzierter Lernrate verhinderten den Kollaps nicht; weitergehende Ansätze wie *Proximal Feature Optimization* (Moalla u. a. o. D.) wurden nicht evaluiert.

Die im Trainingsverlauf ansteigende Landezeit lässt sich auf die Struktur der Belohnungsfunktion zurückführen. Die beiden dominanten Komponenten Fortschritt r_{prog} (Gewicht 1,0) und Nähe r_{nah} (Gewicht 2,0) werden bei steilem Sinkflug maximiert, da die 3D-Distanz zum

Ziel pro Schritt am stärksten abnimmt. Entscheidend ist dabei die unterschiedliche Natur beider Komponenten: $r_{\text{prog}} = d_t - d_{t+1}$ misst die Distanzverringerung pro Schritt und ist damit potentialbasiert im Sinne von Ng u. a. (1999); solches Reward Shaping verändert die optimale Strategie nicht. $r_{\text{nah}} = \exp(-d_{t+1})$ hingegen ist nicht potentialbasiert, sondern ein zustandsabhängiger Pro-Schritt-Reward: Jeder Zeitschritt in Zielnähe liefert Belohnung, unabhängig davon, ob der Agent sich dem Ziel weiter nähert. Da es keine Zeitstrafe gibt und die Erfolgsbelohnung r_{erf} nur einmalig ausgelöst wird, überwiegt der akkumulierte Schweben-Anreiz. Dieses Phänomen ist in der Literatur bekannt: Randlov und Alstrøm (1998) dokumentieren, dass ein nicht-potentialbasierter Shaping-Reward einen Agenten dazu bringt, Kreise zu fahren statt das Ziel zu erreichen, da die akkumulierte Zwischenbelohnung die Zielbelohnung übersteigt. Der Agent lernt folglich eine Strategie, die steilen Abstieg mit ausgedehntem Schweben in Zielnähe kombiniert, was die beobachtete Zunahme der Episodenlänge im Trainingsverlauf erklärt.

6.1.2 Dominante Fehlerart und Flugverhalten

Hinderniskollisionen (28,8 %) sind die einzig relevante Fehlerart; Zeitüberschreitungen und Bodenkollisionen treten nicht auf. Mit oberflächenbasierter Kollisionserkennung (Abschnitt 5.2.1) steigt die Erfolgsrate auf 98,0 %. Die Diskrepanz erklärt sich aus dem Zusammenspiel dreier Faktoren: Erstens begünstigt die Belohnungsstruktur knappes Passieren, da die Hindernisproximität ($w_{\text{prox}} = -0,2$) deutlich schwächer gewichtet ist als Fortschritt und Nähe ($w_{\text{prog}} = 1,0$, $w_{\text{nah}} = 2,0$). Der Agent lernt rational, Hindernisnähe in Kauf zu nehmen, solange er Fortschritt erzielt. Zweitens beschränkt das Entscheidungsintervall von 3 s die Reaktionszeit: Erkennt der Agent ein Hindernis, kann er erst beim nächsten Entscheidungsschritt ausweichen, was bei ungünstigen Konfigurationen nicht ausreicht. Drittens erzwingt die nach unten gerichtete Kamera ein reaktives Flugverhalten, bei dem Hindernisse erst bei Annäherung erkannt werden. Das resultierende Muster (direkter Abstieg, spätes Abbremsen, laterales Ausweichen mit geringem Sicherheitsabstand von im Mittel 0,52 m) ist präzise genug, um die tatsächliche Objektoberfläche zuverlässig zu verfehlen, durchdringt jedoch regelmäßig die konservative AABB-Hülle. Die AABB-Kollision während des Trainings wirkt dabei als implizite Sicherheitsreserve: Der Agent lernt, die größere Begrenzungsbox zu meiden, und hält dadurch zur tatsächlichen Oberfläche mehr Abstand als nötig.

Der trainierte Agent erreicht in der Korridorumgebung eine Erfolgsrate von 71,0 % unter der konservativen AABB-Kollisionsmetrik; mit oberflächenbasierter Kontakterkennung steigt dieser Wert auf 98,0 %. Zur Einordnung: Bartolomei u. a. (2022) berichten vergleichbare

Raten (über 70 %) für ein RL-System mit Tiefenbildeingabe, allerdings für die Selektion sicherer Landezonen in offenem Gelände, nicht für die Navigation durch enge Hindernispassagen. Der Agent erlernt eine reaktive Ausweichstrategie, deren Leistungsgrenze primär durch den begrenzten Sensorhorizont, das Entscheidungsintervall von 3 s und die schwache Gewichtung der Hindernisproximität bestimmt wird. Das beobachtete Flugverhalten legt dabei eine funktionale Rollenverteilung der Eingabemodalitäten nahe: Die lateralen Ausweichmanöver setzen Hindernisinformation voraus, die ausschließlich im Tiefenbild enthalten ist, da der Zustandsvektor keine Hindernispositionen kodiert. Ohne Tiefenbild könnte der Agent lediglich den direkten Abstieg zum Ziel erlernen. Der Zustandsvektor steuert hingegen die Zielnavigation und den kontrollierten Abstieg.

6.2 Transferleistung auf den Weinberg

6.2.1 Leistungsabfall und Verschiebung der Fehlerarten

Der Leistungsabfall von 71,0 % auf 10,4 % ist erwartbar, da der Agent die Weinberggeometrie erstmals zur Evaluationszeit sieht. Aufschlussreich ist die Zusammensetzung: Während in Phase 1 Hinderniskollisionen dominieren (28,8 %), verschiebt sich die Fehlerverteilung in Phase 2 zu Zeitüberschreitungen (73,7 %) bei geringer Terrain-Kollisionsrate (15,9 %). Der Agent erreicht die Zielumgebung (mediane finale Distanz 1,19 m; 59,7 % der Timeout-Episoden innerhalb von 1,5 m), kann dort jedoch nicht konvergieren. Mit erweitertem Erfolgsradius von 1,5 m steigt die Erfolgsrate auf 69,7 %, was dem Phase-1-Niveau entspricht.

Das fehlende Konvergieren ist weder ein Präzisions- noch ein Zeitproblem: In Phase 1 passiert der Agent Hindernisse mit 0,52 m Minimalabstand, und 180 s Zeitlimit bieten circa 60 Entscheidungsschritte. Die Ursache liegt in der strukturell veränderten Landesituation: Im Korridor ist die Landezone hindernisfrei; im Weinberg ragen Reihen direkt in die Landeumgebung, eine Konfiguration, die im Training nie auftrat. Dass die Aufgabe grundsätzlich lösbar ist, zeigt der RRT-Connect-Planer, der im selben Weinberg 85,7 % erreicht und bei erweitertem Erfolgsradius (1,5 m) auf 98,4 % steigt (vgl. Abschnitt 6.3). Der Leistungsabfall ist somit kein Artefakt der Umgebung, sondern eine Eigenschaft des lernbasierten Ansatzes.

6.2.2 Einordnung als Sim-to-Sim Transfer

Die Phase-2-Evaluation ist als Zero-Shot Sim-to-Sim Transfer einzuordnen: Der Agent wird ohne Fine-Tuning und ohne Domain Randomization in einer geometrisch verschiedenartigen

Umgebung evaluiert. Dass die Navigationsleistung dennoch transferiert (69,7 % bei erweitertem Erfolgsradius), lässt sich auf die umgebungsagnostische Beobachtungsstruktur zurückführen: Der Zustandsvektor kodiert relative Positionen, und das Tiefenbild enthält keine texturbasierten Merkmale, die an die Korridorgeometrie gebunden wären. Dass die Hindernisumgehung auch auf zuvor unbekannte Hindernisformen generalisiert (Phase 1), zeigt zudem, dass das CNN geometrische Strukturen erkennt, die über die Trainingsumgebung hinaus transferieren; die Grenzen dieser Repräsentation offenbart das im Folgenden diskutierte Perceptual Aliasing. Sim-to-Real-Arbeiten berichten vergleichbare Einbußen bei Umgebungswechseln, selbst mit Domain Randomization: Polvara u. a. (2020) verlieren 28 Prozentpunkte, Ladosz u. a. (2024) 27 Prozentpunkte. Der hier beobachtete nominale Abfall von 61 Prozentpunkten relativiert sich bei erweitertem Radius auf 1,3 Prozentpunkte, was zeigt, dass der Umgebungswechsel die terminale Landephase betrifft, nicht die Navigation. Sim-to-Sim Transfer unter kontrollierten Bedingungen ist dabei ein schwächeres Ergebnis als Sim-to-Real Transfer oder Generalisierung über diverse Trainingsumgebungen; die Übertragbarkeit sollte daher nicht überbewertet werden.

6.2.3 Perceptual Aliasing als Erklärung für die Terrain-Kollisionen

Ein möglicher Mechanismus hinter den verbleibenden 15,9 % Terrain-Kollisionen ist Perceptual Aliasing im Sinne von Whitehead und Ballard (1991): Die Kamera erzeugt für Boden und Vegetation bei Annäherung ähnliche Signaturen (Abbildung 6.1), sodass unterschiedliche Zustände aus der Wahrnehmung allein nicht unterscheidbar sind. In der Trainingsumgebung ist diese Ambiguität unproblematisch, da die Landezone hindernisfrei ist und der Agent das Tiefenbild in der terminalen Phase gefahrlos ignorieren kann. Im Weinberg bricht diese Strategie: Reihen ragen direkt in die Landezone, sodass ein Agent, der das Tiefenbild in Zielnähe zu ignorieren gelernt hat, mit Vegetation kollidiert. Bartolomei u. a. (2022) adressieren dieses Problem durch zusätzliche semantische Segmentierungen als Eingabe; im vorliegenden System steht eine solche Unterscheidung nicht zur Verfügung.

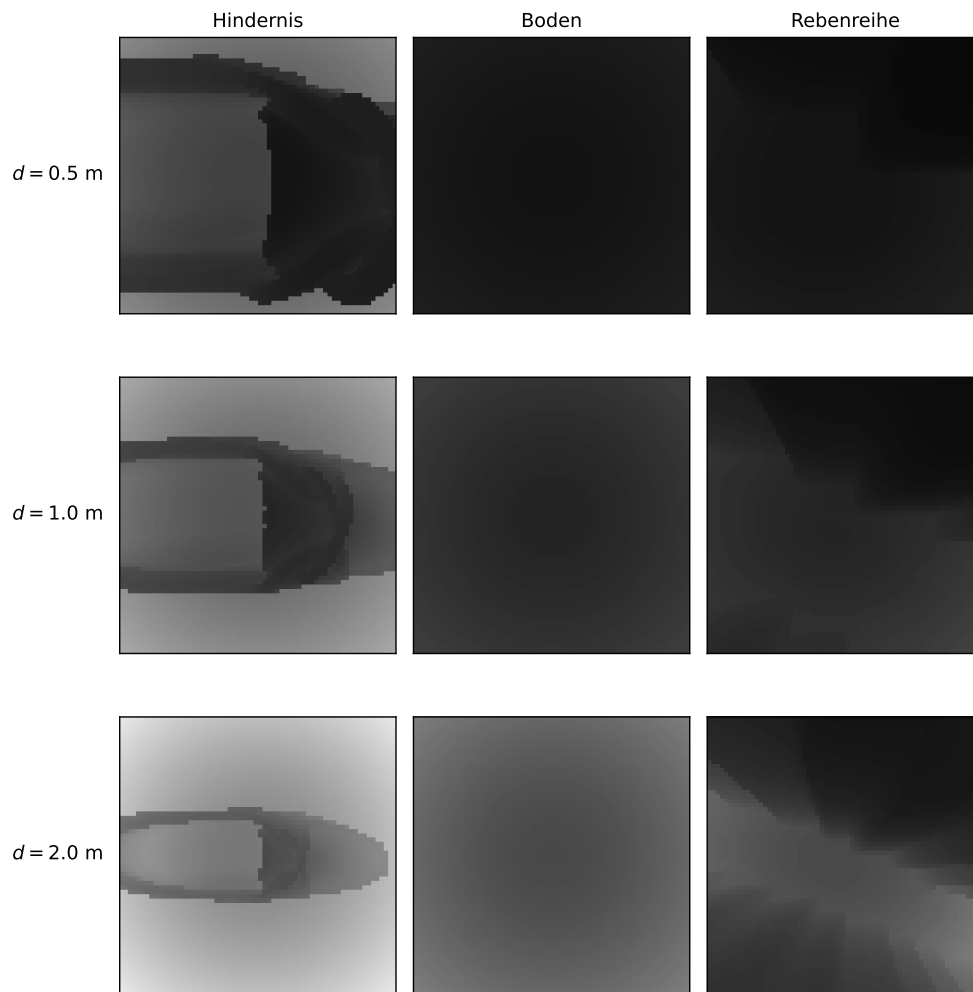


Abbildung 6.1.: Tiefenbilder der nach unten gerichteten Kamera bei drei Distanzen ($d = 0,5, 1,0, 2,0 \text{ m}$). Links: Korridorhindernis mit klar erkennbarer Kontur. Mitte: Boden. Rechts: Rebenreihe im Weinberg. Boden und Rebenreihe erzeugen deutlich ähnliche Signaturen als Korridorhindernisse, deren Konturen bei allen Distanzen klar hervortreten.

Verschärft wird das Problem durch die fehlende zeitliche Gedächtnisstruktur: Der Agent verarbeitet nur das aktuelle Tiefenbild. Erkennt er eine Rebenreihe unter sich und weicht lateral aus, fehlt ihm die Repräsentation benachbarter Reihen außerhalb des nach unten gerichteten Sichtfelds. In der Trainingsumgebung sind Hindernisse isolierte Objekte mit freiem Umgebungsraum; im Weinberg führt laterales Ausweichen systematisch in die nächste Rebenreihe. Singla u. a. (2018) zeigen, dass rekurrente Architekturen mit einem Aufmerksamkeitsmechanismus über vergangene Beobachtungen dieses Problem adressieren können, indem sie vergangene Tiefenbeobachtungen über die Zeit integrieren und so das begrenzte Sichtfeld kompensieren.

Die erlernte Navigationsleistung lässt sich partiell auf die Weinbergumgebung übertragen: Die Erfolgsrate sinkt von 71,0 % auf 10,4 %, wobei sich die dominante Fehlerart von Hinderniskollisionen zu Zeitüberschreitungen verschiebt. Bei erweitertem Erfolgswert (1,5 m)

erreicht der Agent 69,7 %, was zeigt, dass die Hindernisnavigation ohne Fine-Tuning generalisiert. Die terminale Landephase scheitert an der im Training nie erfahrenen Konfiguration von Vegetation in der Landezone, verstärkt durch Perceptual Aliasing zwischen Boden- und Hindernissignaturen im Tiefenbild.

6.3 Leistungslücke zum Pfadplaner

6.3.1 Informationsasymmetrie und Erfolgsrate

Der RRT-Connect-Planer übertrifft den RL-Agenten in der Erfolgsrate beider Umgebungen: um 11 Prozentpunkte im Korridor (82,0 % gegenüber 71,0 %) und um 75 Prozentpunkte im Weinberg (85,7 % gegenüber 10,4 %). Angesichts der in Abschnitt 4.8.2 beschriebenen Informationsasymmetrie quantifiziert dieser Vergleich nicht die algorithmische Überlegenheit des Planers, sondern den Preis minimaler Sensorik. Unter oberflächenbasierter Kollisionserkennung (Abschnitt 6.1) erreicht der RL-Agent im Korridor 98,0 % ($n = 100$) und liegt damit über der Planerleistung von 82,0 % ($n = 1000$). Aufgrund der unterschiedlichen Stichprobengrößen ist dieser Vergleich als vorläufig zu werten.

Das vorliegende System operiert bewusst am unteren Ende des in der Literatur eingesetzten Sensorspektrums. Dass der RL-Agent unter fairer Kollisionsmetrik das Navigationsproblem vergleichbar mit einem vollständig informierten Planer löst, bestätigt den von Semerikov u. a. (2025) beschriebenen Trend, dass lernbasierte Ansätze begrenzte Sensorausstattung algorithmisch kompensieren können. Die Grenzen dieses Prinzips zeigt der Weinberg-Transfer (vgl. Abschnitt 6.2).

6.3.2 Landezeit und Rechenzeit

Der RL-Agent landet in beiden Umgebungen schneller als der RRT-Planer (30,0 s gegenüber 42,0 s im Korridor). Dies erklärt sich durch die in Abschnitt 6.1 beschriebene Flugstrategie: Der Agent lernt direkte Abstiegs trajektorien mit knappem Ausweichen, während der Planer konservativere Pfade mit größeren Sicherheitsabständen berechnet, was sich in der höheren Pfadeffizienz des Planers widerspiegelt (1,09 gegenüber 1,19). Der deutlichste Unterschied zeigt sich in der Rechenzeit: Ein Forward-Pass benötigt 0,21 ms, während RRT-Connect 80,4 ms pro Episode für die Pfadplanung benötigt. Für eingebettete Hardware, wie sie in realen Drohnenanwendungen eingesetzt wird, ist dieser Unterschied ein relevanter praktischer Vorteil.

Die Leistungslücke zum privilegierten RRT-Connect-Planer beträgt 11 Prozentpunkte im Korridor und 75 Prozentpunkte im Weinberg. Im Korridor deutet die oberflächenbasierte Kollisionserkennung (98,0 %, $n = 100$, gegenüber 82,0 %, $n = 1\,000$) darauf hin, dass der lernbasierte Ansatz mit lokaler Sensorik das Navigationsproblem vergleichbar lösen kann; die unterschiedlichen Stichprobengrößen schränken die Belastbarkeit dieses Vergleichs jedoch ein. Der RL-Agent kompensiert die geringere Erfolgsrate durch kürzere Landezeiten und deutlich niedrigere Rechenzeit. Zudem entscheidet der RL-Agent lokal pro Zeitschritt, was ihn im Gegensatz zum global planenden RRT-Connect prinzipiell für dynamische Umgebungen geeignet macht.

6.4 Limitationen

6.4.1 Trainingssetup

Alle Ergebnisse basieren auf einem einzigen Trainingslauf (Seed). Die Bootstrap-Konfidenzintervalle quantifizieren die Evaluationsvarianz über Hinderniskonfigurationen und Startpositionen, nicht die Trainingsvarianz; ein anderer Seed könnte zu qualitativ anderem Verhalten führen. Die Belohnungsgewichte wurden manuell in Vorversuchen abgestimmt, wobei bereits kleine Änderungen das Flugverhalten qualitativ verändern können (vgl. Abschnitt 6.1). Zudem nutzt das Training 512 parallele Umgebungen auf einer einzelnen GPU; eine höhere Parallelisierung kann sowohl die Konvergenzgeschwindigkeit als auch die Strategiequalität verbessern (Xu u. a. 2024).

6.4.2 Sensorik und Architektur

Die AABB-Kollisionserkennung überschätzt das Hindernisvolumen systematisch; die Diskrepanz zwischen 71,0 % (AABB) und 98,0 % (oberflächenbasiert) zeigt, dass die berichtete Erfolgsrate die tatsächliche Navigationsleistung unterschätzt. Wie in Abschnitt 4.5 dargestellt, war die oberflächenbasierte Variante im Training aufgrund des Rechenaufwands nicht einsetzbar; die AABB-Metrik stellt daher einen bewussten Trade-off zwischen Genauigkeit und Trainingseffizienz dar. Der Agent verarbeitet nur das aktuelle Tiefenbild ohne zeitliches Gedächtnis, sodass Strukturen außerhalb des Sichtfelds nicht über mehrere Zeitschritte akkumuliert werden können (vgl. Abschnitt 6.2). Der individuelle Beitrag von Tiefenwahrnehmung und Zustandsvektor wurde nicht durch eine Ablationsstudie quantifiziert; die Schlussfolgerungen über die gelernten CNN-Repräsentationen sind indirekt aus Trajektorien und Transferleistung abgeleitet.

6.4.3 Umgebungsdiversität

Im Training und in Phase 1 ist das Landeziel in allen Episoden identisch; erst die Weinberg-Evaluation nutzt 20 verschiedene Zielpositionen. Die vierfache Skalierung des Weinberg-Modells vergrößert zudem die Reihenabstände relativ zur Drohne. Die Simulation bietet perfekte Sensorik ohne Rauschen, keine Motorverzögerung und keine Kommunikationslatenz; Ergebnisse sind nicht direkt auf reale Drohnen übertragbar.

7 Fazit und Ausblick

7.1 Beantwortung der Forschungsfragen

Die vorliegende Arbeit untersuchte, inwiefern ein RL-basierter Ansatz eine autonome Drohne befähigen kann, mithilfe lokaler Tiefensensorik in hindernisreichen Umgebungen sicher zu landen.

Der trainierte Agent erreicht in der Korridorumgebung eine Erfolgsrate von 71,0 % unter der konservativen AABB-Kollisionsmetrik und 98,0 % unter oberflächenbasierter Kontakterkennung. Er erlernt eine reaktive Ausweichstrategie, deren Leistungsgrenze durch den begrenzten Sensorhorizont, das Entscheidungsintervall und die schwache Gewichtung der Hindernisproximität bestimmt wird (NF1). Die erlernte Navigationsleistung lässt sich partiell auf die Weinbergumgebung übertragen: Die Erfolgsrate sinkt auf 10,4 %, wobei sich die dominante Fehlerart von Hinderniskollisionen zu Zeitüberschreitungen verschiebt. Bei erweitertem Erfolgswert (1,5 m) erreicht der Agent 69,7 %, was zeigt, dass die Hindernisnavigation ohne Fine-Tuning generalisiert; die terminale Landephase scheitert an Perceptual Aliasing zwischen Boden- und Vegetationssignaturen im Tiefenbild (NF2). Die Leistungslücke zum privilegierten RRT-Connect-Planer beträgt 11 Prozentpunkte im Korridor und 75 Prozentpunkte im Weinberg; unter oberflächenbasierter Kollisionserkennung deutet eine vorläufige Nachuntersuchung ($n = 100$) darauf hin, dass diese Lücke im Korridor weitgehend verschwindet. Der RL-Agent kompensiert die geringere Erfolgsrate durch kürzere Landezeiten und deutlich niedrigere Rechenzeit (NF3).

In der Gesamtschau zeigt die Arbeit, dass ein RL-Agent mit minimaler Sensorausstattung hindernisreiche Landemanöver unter kontrollierten Simulationsbedingungen erlernen kann. Eine vorläufige Nachuntersuchung mit oberflächenbasierter Kollisionsmetrik ($n = 100$) deutet darauf hin, dass der Agent das Navigationsproblem im Korridor vergleichbar mit einem vollständig informierten Planer lösen kann, bei Echtzeitfähigkeit und ohne explizite Umgebungsmodellierung. Die Grenzen des Ansatzes liegen im Umgebungstransfer: Perceptual Aliasing und fehlendes zeitliches Gedächtnis begrenzen die Leistung in geometrisch komplexeren Umgebungen.

7.2 Limitationen der Arbeit

Die Ergebnisse basieren auf einem einzigen Trainingslauf, sodass die Trainingsvarianz nicht quantifiziert ist. Die manuell abgestimmten Belohnungsgewichte beeinflussen das Flugverhalten maßgeblich. Der Agent verarbeitet nur das aktuelle Tiefenbild ohne zeitliches Gedächtnis, was die Akkumulation von Strukturen außerhalb des Sichtfelds verhindert. Im Training ist das Landeziel identisch und alle Hindernisse statisch; die Simulation bietet perfekte Sensorik ohne Rauschen, Motorverzögerung oder Kommunikationslatenz, sodass die Ergebnisse nicht direkt auf reale Drohnen übertragbar sind.

7.3 Perspektiven für zukünftige Forschung

7.3.1 Architekturerweiterungen

Eine rekurrente Architektur (LSTM) oder ein Tiefenbild-Stapel würde dem Agenten zeitliches Gedächtnis verleihen und könnte das in Abschnitt 6.2 beschriebene Problem benachbarter Hindernisse außerhalb des Sichtfelds adressieren. Eine Ablationsstudie, in der ein Modell ohne Tiefenbild unter identischen Bedingungen trainiert wird, würde den individuellen Beitrag beider Eingabemodalitäten quantifizieren. Ergänzend ließe eine Visualisierung der CNN-Aktivierungen mittels Grad-CAM direkte Rückschlüsse auf die gelernten Hindernisrepräsentationen zu.

7.3.2 Trainingssetup

Eine Multi-Seed-Validierung ist notwendig, um die statistische Belastbarkeit der Ergebnisse zu erhöhen. Darüber hinaus könnte eine systematische Optimierung der Belohnungsgewichte die manuelle Abstimmung ersetzen und den in Abschnitt 6.1 beschriebenen Schweben-Anreiz gezielt adressieren.

7.3.3 Umgebungsdiversität und Sim-to-Real-Transfer

Domain Randomization, insbesondere Sensorrauschen, Windstörungen und variable Zielpositionen, könnte sowohl die Robustheit als auch den Transfer auf reale Hardware vorbereiten. Die Einführung dynamischer Hindernisse würde zudem prüfen, ob die reaktive, schrittweise Entscheidungsstruktur des RL-Agenten den in Abschnitt 6.3 beschriebenen prinzipiellen Vorteil gegenüber globaler Pfadplanung auch empirisch einlöst. Langfristig stellt der Sim-to-Real-Transfer den entscheidenden nächsten Schritt dar, um die unter kontrollierten Bedingungen gezeigte Leistungsfähigkeit in realen Einsatzszenarien zu validieren. Die vorliegende

Arbeit liefert dafür eine Grundlage, indem sie zeigt, unter welchen Bedingungen lernbasierte Landung mit minimaler Sensorik gelingt und wo ihre Grenzen liegen.

Anhang

Anhangsverzeichnis

Anhang A	Ergänzende Trainingsverläufe	VIII
Anhang B	Phase-1-Evaluationsmetriken	IX
Anhang C	Phase-2-Evaluationsmetriken	X
Anhang D	PID-Reglerparameter	XI

A Ergänzende Trainingsverläufe

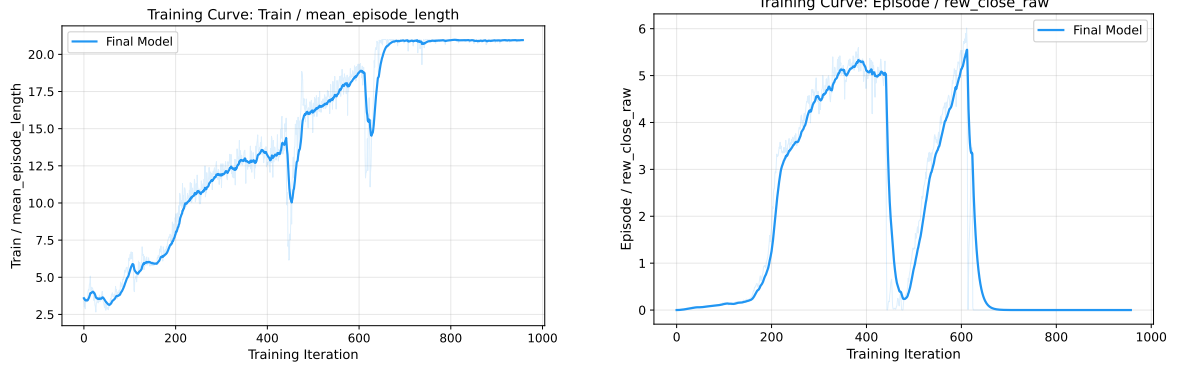


Abbildung A1.: Mittlere Episodenlänge (links) und Näherungsbelohnung (rechts). Die Episodenlänge gibt die Anzahl der Entscheidungsschritte pro Episode an. Die Näherungsbelohnung steigt mit abnehmender Distanz zum Ziel. Geglättet mit EMA $\alpha = 0,9$.

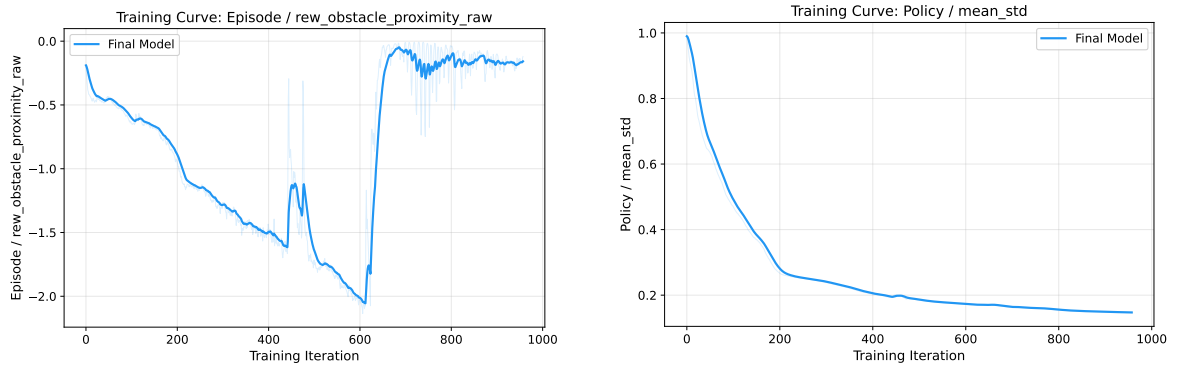


Abbildung A2.: Hindernisabstandsstrafe (links) und mittlere Standardabweichung der Aktionsverteilung (rechts). Geglättet mit EMA $\alpha = 0,9$.

B Phase-1-Evaluationsmetriken

Tabelle B1.: Phase-1-Ergebnisse des RL-Agenten (1 000 Episoden, Checkpoint 400, ungesehene Hindernisse). Konfidenzintervalle: 95 %-Bootstrap (Perzentilmethode, $B = 10\,000$). Landezeit, Pfadeffizienz und Rechenzeit beziehen sich ausschließlich auf erfolgreiche Episoden ($n = 710$).

Metrik	Wert	95 %-KI / IQR
Erfolgsrate	71,0 %	[68,2; 73,8]
Hinderniskollision	28,8 % (288)	[26,0; 31,6]
Verlassen des Korridors	0,2 % (2)	[0,0; 0,5]
Bodenkollision	0,0 % (0)	—
Timeout	0,0 % (0)	—
Landezeit (Median)	30,0 s	IQR [27,0; 36,0]
Pfadeffizienz (Median)	1,19	IQR [1,13; 1,25]
Min. Hindernisabstand (Mittel)	0,52 m	[0,50; 0,54]
Min. Hindernisabstand (global)*	0,00 m	—
Rechenzeit/Episode (Median)	0,21 ms	IQR [0,19; 0,25]

* Stammt aus Episoden, die mit Kollision endeten (Distanz = 0 bei Kontakt).

C Phase-2-Evaluationsmetriken

Tabelle C1.: Phase-2-Ergebnisse des RL-Agenten (1 000 Episoden, Checkpoint 400, Weinberg-Transfer, Erfolgsradius 0,5 m). Konfidenzintervalle: 95 %-Bootstrap (Perzentilmethode, $B = 10\,000$). Landezeit, Pfadeffizienz und Rechenzeit beziehen sich ausschließlich auf erfolgreiche Episoden ($n = 104$).

Metrik	Wert	95 %-KI / IQR
Erfolgsrate	10,4 %	[8,5; 12,4]
Terrain-Kollision	15,9 % (159)	[13,6; 18,2]
Timeout	73,7 % (737)	[71,0; 76,4]
Bodenkollision	entfällt (kein fester Bodenkollisions-Check)	
Verlassen des Korridors	entfällt (kein Korridorbegrenzungsrahmen)	
Landezeit (Median)	36,0 s	IQR [33,0; 36,0]
Pfadeffizienz (Median)	1,07	IQR [1,05; 1,09]
Finale Zieldistanz (Median)	1,19 m	—
Min. Terrain-Abstand (Mittel)	1,02 m	[0,98; 1,06]
Min. Terrain-Abstand (global)*	0,00 m	—
Rechenzeit/Episode (Median)	0,25 ms	IQR [0,23; 0,27]

* Stammt aus Episoden, die mit Terrain-Kollision endeten (Distanz = 0 bei Kontakt).

D PID-Reglerparameter

Tabelle D1.: PID-Verstärkungen des kaskadierenden Reglers.

Regelkreis	K_p	K_i	K_d
Position x/y	1,0	0	0,7
Position z	1,5	0	1,0
Geschwindigkeit x/y	16,0	0	8,0
Geschwindigkeit z	100,0	2,0	10,0
Roll / Nick	6,0	0	3,0
Gier	0,5	0	0,8

Literatur

- Amendola, José, Linga Reddy Cenkeramaddi und Ajit Jha (2024). „Drone Landing and Reinforcement Learning: State-of-Art, Challenges and Opportunities“. In: *IEEE Open Journal of Intelligent Transportation Systems* 5, S. 520–539. ISSN: 2687-7813. DOI: 10.1109/OJITS.2024.3444487. URL: <https://ieeexplore.ieee.org/document/10637701> (besucht am 18.12.2025).
- Arafat, Muhammad Yeasir, Muhammad Morshed Alam und Sangman Moh (27. Jan. 2023). „Vision-Based Navigation Techniques for Unmanned Aerial Vehicles: Review and Challenges“. In: *Drones* 7.2. ISSN: 2504-446X. DOI: 10.3390/drones7020089. URL: <https://www.mdpi.com/2504-446X/7/2/89> (besucht am 13.05.2026).
- Bartolomei, Luca, Yves Kompis, Lucas Teixeira und Margarita Chli (Okt. 2022). „Autonomous Emergency Landing for Multicopters Using Deep Reinforcement Learning“. In: *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), S. 3392–3399. DOI: 10.1109/IROS47612.2022.9981152. URL: <https://ieeexplore.ieee.org/document/9981152> (besucht am 18.12.2025).
- Basso, Bruno und John Antle (Apr. 2020). „Digital Agriculture to Design Sustainable Agricultural Systems“. In: *Nature Sustainability* 3.4, S. 254–256. ISSN: 2398-9629. DOI: 10.1038/s41893-020-0510-0. URL: <https://www.nature.com/articles/s41893-020-0510-0> (besucht am 03.05.2026).
- Botsch, Benny (2023). „Parametrische Methoden“. In: *Maschinelles Lernen - Grundlagen und Anwendungen: Mit Beispielen in Python*. Hrsg. von Benny Botsch. Berlin, Heidelberg: Springer, S. 61–102. ISBN: 978-3-662-67277-8. DOI: 10.1007/978-3-662-67277-8_5. URL: https://doi.org/10.1007/978-3-662-67277-8_5 (besucht am 06.06.2026).
- Calvin, Katherine u. a. (25. Juli 2023). *IPCC, 2023: Climate Change 2023: Synthesis Report. Contribution of Working Groups I, II and III to the Sixth Assessment Report of the*

- Intergovernmental Panel on Climate Change [Core Writing Team, H. Lee and J. Romero (Eds.)]. IPCC, Geneva, Switzerland. Intergovernmental Panel on Climate Change (IPCC). DOI: 10.59327/IPCC/AR6-9789291691647. URL: <https://www.ipcc.ch/report/ar6/syr/> (besucht am 01.04.2026).*
- Climate Change Trends and Impacts on California Agriculture: A Detailed Review | MDPI (2026). URL: <https://www.mdpi.com/2073-4395/8/3/25#Conclusions> (besucht am 01.04.2026).*
- Cobbe, Karl, Jacob Hilton, Oleg Klimov und John Schulman (2021). „Phasic Policy Gradient“. In: *Proceedings of the 38th International Conference on Machine Learning*. Bd. 139. Proceedings of Machine Learning Research. PMLR, S. 2020–2027.
- Coumans, Erwin und Yunfei Bai (2016–2021). *PyBullet, a Python Module for Physics Simulation for Games, Robotics and Machine Learning*. URL: <http://pybullet.org> (besucht am 20.05.2026).
- Downs, Laura, Anthony Francis, Nate Koenig, Brandon Kinman, Ryan Hickman, Krista Reymann, Thomas B. McHugh und Vincent Vanhoucke (25. Apr. 2022). *Google Scanned Objects: A High-Quality Dataset of 3D Scanned Household Items*. DOI: 10.48550/arXiv.2204.11918. arXiv: 2204.11918 [cs.RO]. URL: <http://arxiv.org/abs/2204.11918> (besucht am 23.05.2026). Vorveröffentlichung.
- Duckett, Tom u. a. (2. Aug. 2018). *Agricultural Robotics: The Future of Robotic Agriculture*. DOI: 10.48550/arXiv.1806.06762. arXiv: 1806.06762 [cs]. URL: <http://arxiv.org/abs/1806.06762> (besucht am 03.05.2026). Vorveröffentlichung.
- Efron, Bradley und Robert J. Tibshirani (1994). *An Introduction to the Bootstrap*. noch lesen. New York: Chapman & Hall/CRC. ISBN: 978-0-412-04231-7.
- Eßer, Julian, Gabriel B Margolis, Oliver Urbann, Soren Kerner und Pulkit Agrawal (o. D.). „Action Space Design in Reinforcement Learning for Robot Motor Skills“. In: ().

- Food and Agriculture Organization of the United Nations, Hrsg. (2017). *The Future of Food and Agriculture: Trends and Challenges*. Rome: Food and Agriculture Organization of the United Nations. 163 S. ISBN: 978-92-5-109551-5.
- Genesis Authors (Dez. 2024). *Genesis: A Generative and Universal Physics Engine for Robotics and Beyond*. URL: <https://github.com/Genesis-Embodied-AI/Genesis> (besucht am 20.05.2026).
- Guebsi, Ridha, Sonia Mami und Karem Chokmani (2024). „Drones in Precision Agriculture: A Comprehensive Review of Applications, Technologies, and Challenges“. In: *Drones* 8.11, S. 686. DOI: 10.3390/drones8110686.
- Haarnoja, Tuomas, Aurick Zhou, Pieter Abbeel und Sergey Levine (2018). „Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor“. In: *Proceedings of the 35th International Conference on Machine Learning*. Bd. 80. Proceedings of Machine Learning Research. PMLR, S. 1861–1870. URL: <https://proceedings.mlr.press/v80/haarnoja18b.html>.
- Hodge, Victoria J., Richard Hawkins und Rob Alexander (1. März 2021). „Deep Reinforcement Learning for Drone Navigation Using Sensor Data“. In: *Neural Computing and Applications* 33.6, S. 2015–2033. ISSN: 1433-3058. DOI: 10.1007/s00521-020-05097-x. URL: <https://doi.org/10.1007/s00521-020-05097-x> (besucht am 15.04.2026).
- Hornik, Kurt, Maxwell Stinchcombe und Halbert White (1989). „Multilayer Feedforward Networks Are Universal Approximators“. In: *Neural Networks* 2.5, S. 359–366. ISSN: 0893-6080. DOI: 10.1016/0893-6080(89)90020-8.
- Hultgren, Andrew, Tamma Carleton, Michael Delgado, Diana R. Gergel, Michael Greenstone, Trevor Houser, Solomon Hsiang, Amir Jina, Robert E. Kopp, Steven B. Malevich, Kelly E. McCusker, Terin Mayer, Ishan Nath, James Rising, Ashwin Rode und Jiacan Yuan (Juni 2025). „Impacts of Climate Change on Global Agriculture Accounting for Adaptation“. In: *Nature* 642.8068, S. 644–652. ISSN: 1476-4687. DOI: 10.1038/s41586-025-09085-w. URL: <https://www.nature.com/articles/s41586-025-09085-w> (besucht am 01.04.2026).

- Jarraya, Imen, Abdulrahman Al-Batati, Muhammad Bilal Kadri, Mohamed Abdelkader, Adel Ammar, Wadii Boulila und Anis Koubaa (7. Apr. 2025). „Gnss-Denied Unmanned Aerial Vehicle Navigation: Analyzing Computational Complexity, Sensor Fusion, and Localization Methodologies“. In: *Satellite Navigation* 6.1, S. 9. ISSN: 2662-1363. DOI: 10.1186/s43020-025-00162-z. URL: <https://doi.org/10.1186/s43020-025-00162-z> (besucht am 14.05.2026).
- Kanellakis, Christoforos und George Nikolakopoulos (1. Juli 2017). „Survey on Computer Vision for UAVs: Current Developments and Trends“. In: *Journal of Intelligent & Robotic Systems* 87.1, S. 141–168. ISSN: 1573-0409. DOI: 10.1007/s10846-017-0483-z. URL: <https://doi.org/10.1007/s10846-017-0483-z> (besucht am 13.05.2026).
- Kaufmann, Elia, Leonard Bauersfeld, Antonio Loquercio, Matthias Müller, Vladlen Koltun und Davide Scaramuzza (Aug. 2023). „Champion-Level Drone Racing Using Deep Reinforcement Learning“. In: *Nature* 620.7976, S. 982–987. ISSN: 1476-4687. DOI: 10.1038/s41586-023-06419-4. URL: <https://www.nature.com/articles/s41586-023-06419-4> (besucht am 18.12.2025).
- Kaufmann, Elia, Leonard Bauersfeld und Davide Scaramuzza (22. Feb. 2022). *A Benchmark Comparison of Learned Control Policies for Agile Quadrotor Flight*. arXiv.org. URL: <https://arxiv.org/abs/2202.10796v1> (besucht am 03.06.2026).
- Kim, Minwoo, Jongyun Kim, Minjae Jung und Hyondong Oh (15. Juli 2022). „Towards Monocular Vision-Based Autonomous Flight through Deep Reinforcement Learning“. In: *Expert Systems with Applications* 198, S. 116742. ISSN: 0957-4174. DOI: 10.1016/j.eswa.2022.116742. URL: <https://www.sciencedirect.com/science/article/pii/S0957417422002111> (besucht am 16.05.2026).
- Koenig, Nathan und Andrew Howard (2004). „Design and Use Paradigms for Gazebo, an Open-Source Multi-Robot Simulator“. In: *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. Bd. 3, S. 2149–2154. DOI: 10.1109/IROS.2004.1389727.
- Ladosz, Pawel, Meraj Mammadov, Heejung Shin, Woojae Shin und Hyondong Oh (Mai 2024). „Autonomous Landing on a Moving Platform Using Vision-Based Deep Re-

- inforcement Learning“. In: *IEEE Robotics and Automation Letters* 9.5, S. 4575–4582. ISSN: 2377-3766. DOI: 10.1109/LRA.2024.3379837. URL: <https://ieeexplore.ieee.org/document/10476692/> (besucht am 14.05.2026).
- LeCun, Yann, Yoshua Bengio und Geoffrey Hinton (Mai 2015). „Deep Learning“. In: *Nature* 521.7553, S. 436–444. ISSN: 1476-4687. DOI: 10.1038/nature14539. URL: <https://www.nature.com/articles/nature14539> (besucht am 06.06.2026).
- Loquercio, Antonio, Elia Kaufmann, René Ranftl, Matthias Müller, Vladlen Koltun und Davide Scaramuzza (6. Okt. 2021). „Learning High-Speed Flight in the Wild“. In: *Science Robotics* 6.59, eabg5810. DOI: 10.1126/scirobotics.abg5810. URL: <https://www.science.org/doi/10.1126/scirobotics.abg5810> (besucht am 03.05.2026).
- Makoviychuk, Viktor, Lukasz Wawrzyniak, Yunrong Guo, Michelle Lu, Kier Storey, Miles Macklin, David Hoeller, Nikita Rudin, Arthur Allshire, Ankur Handa und Gavriel State (2021). „Isaac Gym: High Performance GPU-Based Physics Simulation for Robot Learning“. In: *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2)*. URL: https://openreview.net/forum?id=fgFBtYgJQX_.
- Martini, Mauro, Simone Cerrato, Francesco Salvetti, Simone Angarano und Marcello Chia-berge (Aug. 2022). „Position-Agnostic Autonomous Navigation in Vineyards with Deep Reinforcement Learning“. In: *2022 IEEE 18th International Conference on Automation Science and Engineering (CASE)*. 2022 IEEE 18th International Conference on Automation Science and Engineering (CASE), S. 477–484. DOI: 10.1109/CASE49997.2022.9926582. URL: <https://ieeexplore.ieee.org/abstract/document/9926582> (besucht am 03.06.2026).
- Mashori, Abdul Sattar, Fuzhong Li, Muhammad Aman, Wuping Zhang, Shujie Jia, Aamir Ali, Nida Jabeen, Wangli Hao und Yuqiao Yan (1. Aug. 2026). „Remote Sensing through UAVs for Precision Agriculture: Applications, Technical Foundations, Current Barriers, and Future Opportunities“. In: *Smart Agricultural Technology* 14, S. 102074. ISSN: 2772-3755. DOI: 10.1016/j.atech.2026.102074. URL: <https://www.>

sciencedirect.com/science/article/pii/S2772375526002959
(besucht am 03.05.2026).

Meier, Lorenz, Dominik Honegger und Marc Pollefeys (Mai 2015). „PX4: A Node-Based Multithreaded Open Source Robotics Framework for Deeply Embedded Platforms“. In: *2015 IEEE International Conference on Robotics and Automation (ICRA)*. 2015 IEEE International Conference on Robotics and Automation (ICRA). Seattle, WA, USA: IEEE, S. 6235–6240. ISBN: 978-1-4799-6923-4. DOI: 10.1109/ICRA.2015.7140074. URL: <http://ieeexplore.ieee.org/document/7140074/> (besucht am 14.05.2026).

Mnih, Volodymyr, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharmashan Kumaran, Daan Wierstra, Shane Legg und Demis Hassabis (Feb. 2015). „Human-Level Control through Deep Reinforcement Learning“. In: *Nature* 518.7540, S. 529–533. ISSN: 1476-4687. DOI: 10.1038/nature14236. URL: <https://www.nature.com/articles/nature14236> (besucht am 03.05.2026).

Moalla, Skander, Andrea Miele, Daniil Pyatko, Razvan Pascanu und Caglar Gulcehre (o. D.). „No Representation, No Trust: Connecting Representation, Collapse, and Trust Issues in PPO“. In: ().

Ng, Andrew Y., Daishi Harada und Stuart J. Russell (27. Juni 1999). „Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping“. In: *Proceedings of the Sixteenth International Conference on Machine Learning. ICML '99*. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., S. 278–287. ISBN: 978-1-55860-612-8.

Patrik, Aurello, Gaudi Utama, Alexander Agung Santoso Gunawan, Andry Chowanda, Jarot S. Suroso, Rizatus Shofiyanti und Widodo Budiharto (Dez. 2019). „GNSS-based Navigation Systems of Autonomous Drone for Delivering Items“. In: *Journal of Big Data* 6.1, S. 53. ISSN: 2196-1115. DOI: 10.1186/s40537-019-0214-3. URL: <https://journalofbigdata.springeropen.com/articles/10.1186/s40537-019-0214-3> (besucht am 13.05.2026).

- Peter, Robinroy, Lavanya Ratnabala, Demetros Aschu, Aleksey Fedoseev und Dzmitry Tsetserukou (25. Juni 2024). *TornadoDrone: Bio-inspired DRL-based Drone Landing on 6D Platform with Wind Force Disturbances*. Version 2. DOI: 10.48550/arXiv.2406.16164. arXiv: 2406.16164 [cs]. URL: <http://arxiv.org/abs/2406.16164> (besucht am 07.03.2026). Vorveröffentlichung.
- Polvara, Riccardo, Massimiliano Patacchiola, Marc Hanheide und Gerhard Neumann (25. Feb. 2020). „Sim-to-Real Quadrotor Landing via Sequential Deep Q-Networks and Domain Randomization“. In: *Robotics* 9.1. ISSN: 2218-6581. DOI: 10.3390/robotics9010008. URL: <https://www.mdpi.com/2218-6581/9/1/8> (besucht am 07.02.2026).
- Precision Landing* (2026). PX4 Autopilot. URL: https://docs.px4.io/main/en/advanced_features/precland (besucht am 14.05.2026).
- Radoglou-Grammatikis, Panagiotis, Panagiotis Sarigiannidis, Thomas Lagkas und Ioannis Moscholios (8. Mai 2020). „A Compilation of UAV Applications for Precision Agriculture“. In: *Computer Networks* 172, S. 107148. ISSN: 1389-1286. DOI: 10.1016/j.comnet.2020.107148. URL: <https://www.sciencedirect.com/science/article/pii/S138912862030116X> (besucht am 03.05.2026).
- Randlov, Jette und Preben Alstrøm (1. Jan. 1998). *Learning to Drive a Bicycle Using Reinforcement Learning and Shaping*. S. 471. 463 S.
- Rudin, Nikita, David Hoeller, Philipp Reist und Marco Hutter (2022). „Learning to Walk in Minutes Using Massively Parallel Deep Reinforcement Learning“. In: *Proceedings of the 5th Conference on Robot Learning*. Bd. 164. Proceedings of Machine Learning Research. PMLR, S. 91–100.
- Schulman, John, Sergey Levine, Philipp Moritz, Michael I. Jordan und Pieter Abbeel (2015). „Trust Region Policy Optimization“. In: *Proceedings of the 32nd International Conference on Machine Learning (ICML)*. Bd. 37. Proceedings of Machine Learning Research. PMLR, S. 1889–1897. URL: <https://proceedings.mlr.press/v37/schulman15.html>.
- Schulman, John, Philipp Moritz, Sergey Levine, Michael I. Jordan und Pieter Abbeel (2016). „High-Dimensional Continuous Control Using Generalized Advantage Estimation“.

- In: *4th International Conference on Learning Representations (ICLR)*. URL: <https://arxiv.org/abs/1506.02438>.
- Schulman, John, Filip Wolski, Prafulla Dhariwal, Alec Radford und Oleg Klimov (28. Aug. 2017). *Proximal Policy Optimization Algorithms*. arXiv: 1707.06347 [cs.LG]. URL: <https://arxiv.org/abs/1707.06347>.
- Semerikov, Serhiy O., Pavlo P. Nechypurenko, Tetiana A. Vakaliuk, Iryna S. Mintii und Andrii O. Kolhatin (28. Okt. 2025). „Vision-Based Autonomous UAV Landing: A Comprehensive Review of Technologies, Techniques, and Applications“. In: *Journal of Intelligent & Robotic Systems* 111.4, S. 115. ISSN: 1573-0409. DOI: 10.1007/s10846-025-02314-4. URL: <https://doi.org/10.1007/s10846-025-02314-4> (besucht am 05.05.2026).
- Singla, Abhik, Sindhu Padakandla und Shalabh Bhatnagar (8. Nov. 2018). *Memory-Based Deep Reinforcement Learning for Obstacle Avoidance in UAV with Limited Environment Knowledge*. arXiv.org. URL: <https://arxiv.org/abs/1811.03307v1> (besucht am 30.05.2026).
- Sutton, Richard S. und Andrew Barto (2020). *Reinforcement Learning: An Introduction*. Second edition. Adaptive Computation and Machine Learning. Cambridge, Massachusetts London, England: The MIT Press. 526 S. ISBN: 978-0-262-03924-6.
- Tavasci, Luca, Francesco Nex und Stefano Gandolfi (20. Sep. 2024). „Reliability of Real-Time Kinematic (RTK) Positioning for Low-Cost Drones' Navigation across Global Navigation Satellite System (GNSS) Critical Environments“. In: *Sensors* 24.18. ISSN: 1424-8220. DOI: 10.3390/s24186096. URL: <https://www.mdpi.com/1424-8220/24/18/6096> (besucht am 14.05.2026).
- Tayar, Marco S., Lucas K. de Oliveira, Felipe Andrade G. Tommaselli, Juliano D. Negri, Thiago H. Segreto, Ricardo V. Godoy und Marcelo Becker (22. Aug. 2025). *Autonomous UAV Flight Navigation in Confined Spaces: A Reinforcement Learning Approach*. arXiv.org. DOI: 10.1109/LARS69345.2025.11273007. URL: <https://arxiv.org/abs/2508.16807v2> (besucht am 03.06.2026).

- Unde, Sanket S., V. K. Kurkute, Sachin S. Chavan, Dadaso D. Mohite, Akshay A. Harale und Ayaan Chougale (16. Sep. 2025). „The Expanding Role of Multirotor UAVs in Precision Agriculture with Applications AI Integration and Future Prospects“. In: *Discover Mechanical Engineering* 4.1, S. 38. ISSN: 2731-6564. DOI: 10.1007/s44245-025-00132-4. URL: <https://doi.org/10.1007/s44245-025-00132-4> (besucht am 29.01.2026).
- Voos, Holger und Haitham Bou-Ammar (Sep. 2010). „Nonlinear Tracking and Landing Controller for Quadrotor Aerial Robots“. In: *2010 IEEE International Conference on Control Applications*. Control (MSC). Yokohama, Japan: IEEE, S. 2136–2141. ISBN: 978-1-4244-5362-7. DOI: 10.1109/CCA.2010.5611204. URL: <http://ieeexplore.ieee.org/document/5611204/> (besucht am 13.05.2026).
- Wang, Junqiao, Zhongliang Yu, Dong Zhou, Jiaqi Shi und Runran Deng (9. Dez. 2024). *Vision-Based Deep Reinforcement Learning of UAV Autonomous Navigation Using Privileged Information*. Version 1. DOI: 10.48550/arXiv.2412.06313. arXiv: 2412.06313 [cs]. URL: <http://arxiv.org/abs/2412.06313> (besucht am 03.04.2026). Vorveröffentlichung.
- Wei, Peng, Prabhash Ragbir, Stavros G. Vougioukas und Zhaodan Kong (1. Nov. 2025). „Vision-Based Navigation of Unmanned Aerial Vehicles in Orchards: An Imitation Learning Approach“. In: *Computers and Electronics in Agriculture* 238, S. 110802. ISSN: 0168-1699. DOI: 10.1016/j.compag.2025.110802. URL: <https://www.sciencedirect.com/science/article/pii/S0168169925009081> (besucht am 03.06.2026).
- Whitehead, Steven D. und Dana H. Ballard (1. Juli 1991). „Learning to Perceive and Act by Trial and Error“. In: *Machine Learning* 7.1, S. 45–83. ISSN: 1573-0565. DOI: 10.1023/A:1022619109594. URL: <https://doi.org/10.1023/A:1022619109594> (besucht am 30.05.2026).
- World Population Prospects* (2026). URL: <https://population.un.org/wpp/> (besucht am 03.05.2026).
- Wubben, Jamie, Francisco Fabra, Carlos T. Calafate, Tomasz Krzeszowski, Johann M. Marquez-Barja, Juan-Carlos Cano und Pietro Manzoni (12. Dez. 2019). „Accura-

te Landing of Unmanned Aerial Vehicles Using Ground Pattern Recognition“. In: *Electronics* 8.12. ISSN: 2079-9292. DOI: 10.3390/electronics8121532. URL: <https://www.mdpi.com/2079-9292/8/12/1532> (besucht am 14.05.2026).

Xu, Zhefan, Xinming Han, Haoyu Shen, Hanyu Jin und Kenji Shimada (24. Sep. 2024). „NavRL: Learning Safe Flight in Dynamic Environments“. In: *arXiv preprint*. DOI: 10.48550/arXiv.2409.15634. arXiv: 2409.15634 [cs.RO].

Yang, Tao, Peiqi Li, Huiming Zhang, Jing Li und Zhi Li (15. Mai 2018). „Monocular Vision SLAM-Based UAV Autonomous Landing in Emergencies and Unknown Environments“. In: *Electronics* 7.5. ISSN: 2079-9292. DOI: 10.3390/electronics7050073. URL: <https://www.mdpi.com/2079-9292/7/5/73> (besucht am 07.02.2026).

Erklärung

Ich versichere, dass ich die vorliegende Arbeit mit dem Thema:

„Titel“

selbständig und nur unter Verwendung der angegebenen Quellen und Hilfsmittel angefertigt habe, insbesondere sind wörtliche oder sinngemäße Zitate als solche gekennzeichnet. Mir ist bekannt, dass Zuwiderhandlung auch nachträglich zur Aberkennung des Abschlusses führen kann. Ich versichere, dass das elektronische Exemplar mit den gedruckten Exemplaren übereinstimmt.

Leipzig, den Datum

AUTORENNAME